

ROSOrin Pro 활용 자율주행 로봇 프로그래밍

디자인: 하이브리드형 · 트랙: 대학교

SV로보틱스

목차

- **00-2** — 전원 인가 및 초기 구동
- **02-1** — ROS2 첫걸음
- **03-1** — 첫 이동 명령
- **03-2** — 바퀴 전방향 테스트
- **06-1** — 라인트레이싱 색상 캘리브레이션
- **07-1** — 물체 색상 추적
- **08-1** — 라이다 장애물 회피
- **09-1** — 로봇팔 첫 동작
- **10-1** — YOLO 물체 인식
- **11-1** — 음성으로 로봇 움직이기
- **12-1** — LLM 기반 자연어 이동 제어
- **13-1** — 쓰레기 분류 로봇
- **14-1** — 색상+AprilTag 물체 분류
- **15-1** — 손 제스처로 로봇 제어하기
- **16-1** — 자율주행 통합
- **17-1** — LLM 함수 호출(function calling)
- **18-1** — LLM 색상 선택 라인트레이싱
- **19-1** — 낙상 감지 로봇
- **20-1** — 사람 따라가기
- **21-1** — 지정한 사람만 따라 움직이기

02. 전원 인가 및 초기 구동

이 레슨이 끝나면

- ✓ 배터리를 안전하게 연결하고 규정된 순서로 충전할 수 있다.
- ✓ 정상 부팅 여부를 LED·부저(buzzer) 신호로 판별할 수 있다.
- ✓ 부팅 이상 발생 시 SD카드 접촉 불량 등 기본 원인을 진단하고 조치할 수 있다.

준비물

조립 매뉴얼(사진자료 2·7p), 배터리 커넥터, 전용 충전기

M4×6 나사 ×4(로봇팔), M3×6 나사 ×4(카메라 브라켓), 내육각 렌치

⚠ 안전 유의사항

- 전원 스위치가 **꺼진 상태**에서만 배터리 커넥터를 연결한다.
- 적-적, 흑-흑 극성(polarity)을 확인한다. 역결선 금지.
- 충전기는 반드시 배터리 DC 포트에 연결한다 — Jetson 보드 DC 입력 절대 금지.
- 배터리 전압이 10V 미만이면 부저 경고음이 울린다. 즉시 충전한다.

수행 순서

- 1 로봇팔 → 덤스카메라 → (옵션)음성박스 → (옵션)LCD 순서로 조립
나사 8개소 체결 완료
- 2 전원 OFF 확인 후 배터리 커넥터 연결
적-적/흑-흑 일치
- 3 충전기를 배터리 DC 포트에 연결, 표시등 확인
표시등 녹색 전환
- 4 전원 스위치 ON, 약 1분 대기
LED1 파란 점멸 + 부저 1회

자가 점검

평가준거	평가방법	피드백 방법
배터리 극성을 정확히 연결하였는가	육안 확인	즉시 교정 지도
정상 부팅 신호(LED·부저)를 확인하였는가	실습 관찰	재시도 안내
부팅 이상 시 조치 절차를 서술할 수 있는가	서술형 확인	구두 피드백

심화 개념

- 리튬 배터리는 과충전·과방전에 민감해서 전용 충전기가 전압을 정밀 제어한다 — 일반 USB 충전기나 다른 전압 어댑터를 쓰면 안 되는 이유다.
- LED·부저(buzzer) 신호 체계는 STM32 컨트롤러가 관리하며, 이 신호를 알아두면 이후 모든 모듈에서 "로봇이 정상인지"를 빠르게 판별할 수 있다.

트러블슈팅

증상	원인	해결
충전기 연결해도 표시등이 안 켜짐	콘센트 미연결 또는 충전기 불량	콘센트 연결 확인 후 다른 콘센트로 재시도
전원 ON 했는데 부저만 울리고 LED는 안 켜짐	배터리 전압 10V 미만(저전압 경고)	즉시 충전 후 재시도
SD카드 재장착해도 계속 무반응	SD카드 자체 손상 가능성	제조사 기술지원 문의

개념 확인

문 항	질문	답
1	배터리 커넥터를 반대로(역극성으로) 연결하면 어떻게 되는가?	_____
2	충전 중 표시등 색깔과, 완료됐을 때 표시등 색깔은 각각 무엇인가?	_____
3	부저가 "삐-삐-삐" 반복해서 울리는 것은 무엇을 의미하는가?	_____

심화 과제 — 로봇이 배터리 전압을 ROS2 토픽으로 실시간 발행하는지 `ros2 topic list`로 직접 찾아보고, 있다면 `ros2 topic echo`로 현재 전압을 확인해보세요. 정답을 미리 알려주지 않는 것은 의도한 것입니다 — 스스로 찾는 연습입니다.

02-1. ROS2 첫걸음 — 노드와 토픽

이 레슨이 끝나면

- ✓ ROS2 작업환경을 활성화하고 현재 실행 중인 노드·토픽 목록을 확인할 수 있다.
- ✓ 최소 구성의 rclpy 노드를 작성하고 실행할 수 있다.
- ✓ 실행 로그와 `ros2 node list`로 노드의 정상 동작 여부를 확인할 수 있다.

준비물

없음 (소프트웨어 실습)

SSH 접속 가능한 PC, ROSOrin Pro 로봇, 터미널 2개

선수지식 더 보기

- [ROS2 노드·토픽 개념](#)

토픽 개념 설명 부분 위주 — 글 안에서 저자가 이전 글(노드 개념)도 함께 요약 언급하므로 별도로 안 찾아봐도 됨

— PinkWink 블로그 (국내 로보틱스 교육자, 한국어)

⚠ 안전 유의사항

- 새 터미널을 열 때마다 `source /opt/ros/humble/setup.zsh; source ~/ros2_ws/install/setup.zsh`를 반드시 실행한다.
- 노드 이름은 로봇에서 이미 실행 중인 시스템 노드와 겹치지 않게 짓는다.

수행 순서

- 1 **로봇에 SSH 접속 후 작업환경 source**
ros2 명령 정상 인식
- 2 **ros2 topic list로 현재 토픽 확인**
/controller/cmd_vel 등 출력
- 3 **hello_robot.py 작성 (아래 코드)**
코드 저장 완료
- 4 **실행 후 별도 터미널에서 ros2 node list 확인**
목록에 노드 이름 표시

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class HelloRobot(Node):
    def __init__(self):
        super().__init__('hello_robot')
        self.create_timer(1.0, self.tick)
        self.count = 0

    def tick(self):
        self.count += 1
        self.get_logger().info(f'ROSOrin Pro 노드 실행 중... {self.count}')

def main():
    rclpy.init()
    rclpy.spin(HelloRobot())

if __name__ == '__main__':
    main()
```

자가 점검

평가준거	평가방법	피드백 방법
작업환경 source를 스스로 수행하는가	실습 관찰	즉시 재안내
노드 실행 로그가 정상 출력되는가	실습 관찰	코드 리뷰
ros2 node list에 자신의 노드가 표시되는가	실습 관찰	재실행 안내

심화 개념

- 노드(Node)는 프로세스 하나에 대응하고, 토픽(Topic)은 발행-구독(pub-sub) 방식으로 메시지가 흐르는 통로다. 하나의 토픽에 여러 노드가 동시에 구독할 수 있다.
- `ros2 node list`, `ros2 topic echo <토픽>`, `ros2 topic hz <토픽>`는 실행 중인 시스템을 진단하는 기본 CLI 도구다.
- `rclpy.spin()`은 노드를 계속 실행 상태로 유지하며 타이머·구독 콜백을 처리한다 — 이게 없으면 타이머가 한 번도 실행되지 않는다.

트러블슈팅

증상	원인	해결
ros2: command not found 노드를 실행했는데 로그가 전혀 안 뜸	작업환경(source)을 안 함 <code>rclpy.spin()</code> 호출을 빠뜨림	source 두 줄을 다시 실행 <code>main()</code> 함수에 <code>spin</code> 추가
ros2 node list에 내 노드가 안 보임	노드가 이미 종료됐거나 에러로 죽음	터미널에 남은 에러 메시지 확인

개념 확인

문항	질문	답
1	노드와 토픽의 차이를 한 문장으로 설명하시오.	_____
2	<code>rclpy.spin()</code> 을 빼먹으면 어떤 문제가 생기는가?	_____
3	작업환경(source)은 왜 매 터미널마다 새로 실행해야 하는가?	_____

심화 과제 — `create_timer`의 주기를 1.0에서 0.5로 바꿔서 로그 출력 빈도가 어떻게 달라지는지 관찰해보세요. 그다음 두 번째 타이머를 추가해서 서로 다른 주기로 다른 메시지를 출력하게 만들어보세요.

03-1. 첫 이동 명령 — cmd_vel로 로봇 움직이기

이 레슨이 끝나면

- ✓ Twist 메시지의 구조(linear.x, angular.z)를 이해하고 로봇을 전진(forward)·회전(rotation) 시킬 수 있다.
- ✓ 1회성 명령 발행이 유실될 수 있는 이유를 알고, 안전한 반복 발행(repeated publish) 방식을 적용할 수 있다.

준비물

없음 (소프트웨어 실습)

ROSOrin Pro, 전후좌우 1.5m 이상 빈 공간

⚠ 안전 유의사항

- 실습 전 로봇 주변 1.5m 이상 공간을 확보한다.
- 정지 명령도 1회성이 아닌 반복 발행으로 보낸다.
- 처음에는 저속(0.2m/s 이하)으로 시작한다.

수행 순서

- 1 **CLI로 반복 발행 체험:** `ros2 topic pub -r 10 /controller/cmd_vel ...`
로봇이 전진함
- 2 **Ctrl+C 후 별도로 정지 명령(0,0) 발행**
로봇이 완전히 정지함
- 3 **first_move.py 작성 (아래 코드) — 5초 전진 후 정지**
코드 저장 완료
- 4 **실행 후 관찰**
5초간 전진 후 정지

```
#!/usr/bin/env python3
import time, rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class FirstMove(Node):
    def __init__(self):
        super().__init__('first_move')
        self.pub = self.create_publisher(Twist, '/controller/cmd_vel', 1)

    def move_forward(self, seconds=5.0, speed=0.2):
        twist = Twist(); twist.linear.x = speed
        end = time.time() + seconds
        while time.time() < end:
            self.pub.publish(twist)    # 반복 발행 — --once 유실 방지
            time.sleep(0.1)
        self.pub.publish(Twist())      # 정지도 발행으로

def main():
    rclpy.init()
    FirstMove().move_forward()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

자가 점검

평가준거	평가방법	피드백 방법
Twist 메시지 구조를 설명할 수 있는가	구두 확인	보충 설명
로봇이 지시한 방향·시간대로 움직였는가	실습 관찰	재시도 안내
반복 발행이 필요한 이유를 서술할 수 있는가	서술형 확인	구두 피드백

심화 개념

- Twist 메시지는 $\text{linear}(x,y,z) \cdot \text{angular}(x,y,z)$ 총 6개 값을 갖지만, 지상 바퀴로봇은 보통 $\text{linear.x} \cdot \text{linear.y} \cdot \text{angular.z}$ 3개만 쓴다 — z축 이동이나 x/y축 회전은 물리적으로 불가능하기 때문이다.
- 퍼블리셔의 큐 크기(`create_publisher` 세 번째 인자)는 메시지가 밀릴 때 몇 개까지 버퍼링할지 정한다. 이동 제어처럼 "최신 값만 중요한" 경우 작게(1) 유지하는 게 일반적이다.

트러블슈팅

증상	원인	해결
--once로 보냈는데 로봇이 안 움직임	발행이 구독자와 매칭되기 전에 유실 (실측 확인된 버그)	-r 10으로 반복 발행
코드 실행 후 로봇이 계속 움직임(안 멈춤)	정지 코드가 실행되지 않고 프로그램이 강제 종료됨	try/finally로 정지 코드 실행을 보장
속도값을 크게 줬는데 이상하게 움직임	권장 범위(-0.6~0.6 m/s) 초과	범위 내 값으로 조정

개념 확인

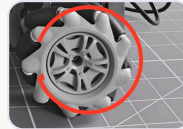
문항	질문	답
1	--once가 유실될 수 있는 이유는 무엇인가?	_____
2	linear.x가 음수면 로봇은 어떻게 움직이는가?	_____
3	정지 명령도 반복 발행해야 하는 이유는 무엇인가?	_____

심화 과제 — move_forward 함수를 확장해서 "사각형 경로로 주행하기"(전진 → 회전(rotation) 90도 → 이 과정을 4회 반복)를 구현해보세요.

03-2. 바퀴 전방향 테스트 — 전진(forward)·평행이동(lateral translation)·회전(rotation) 모두 확인하기



전진(forward)
회전



후진
(backward) 회
전

바퀴 회전 방향 ↔ 로봇 진행 방향 — 네 바퀴 모두 사진과 같은 방향으로 돌면 (왼쪽) 로봇은 앞으로, 반대 방향으로 돌면(오른쪽) 뒤로 갑니다. 이 관계는 명확합니다. 반면 **평행이동(lateral translation)**이나 **제자리 회전(rotation)**은 바퀴마다 서로 다른 방향으로 돌아야 나오는 조합 동작이라, 어떤 바퀴가 어느 방향으로 돌아야 어느 쪽으로 가는지는 로봇마다·문서마다 표기가 갈릴 수 있어 여기서 단정하지 않았습니다 — 오늘 실습에서 `linear.y`·`angular.z`를 직접 넣어보고 실제 방향을 관찰·기록하는 것이 바로 이 부분을 확인하는 과정입니다.

이 레슨이 끝나면

- ✓ Twist 메시지의 `linear.x`·`linear.y`·`angular.z`를 조합해 로봇을 여러 방향으로 움직일 수 있다.
- ✓ 각 축의 부호(+/-)가 실제로 어느 방향에 대응하는지 직접 실험으로 확인하고 기록할 수 있다.
- ✓ 전진·후진·좌우 평행이동·좌우회전 6가지 이동을 모두 시도하고 결과를 기록으로 남길 수 있다.

준비물

없음 (소프트웨어 실습)

ROSOrin Pro, 전후좌우 대각선 포함 1.5m 이상 여유공간

선수지식 더 보기

- [메카닉 휠 역기구학\(수식 유도\)](#)

페이지 중반 'Inverse kinematics' 섹션만 보면 됨 — $v_x \cdot v_y \cdot \omega$ 를 4개 바퀴 속도로 바꾸는 행렬식과 대각선 이동 계산 예시가 나옴. 앞부분(휠 구조 설명)은 건너뛰어도 무방

— Ecam Eurobot (로봇 경진대회 팀의 오픈 튜토리얼, 영문)

⚠ 안전 유의사항

- 실습 전 로봇 전후좌우(대각선 포함) 1.5m 이상 공간을 확보한다.
- 모든 테스트는 저속(0.2 m/s 이하)으로 시작한다.
- 정지 명령도 1회성이 아닌 반복 발행으로 보낸다.
- 대각선 이동(전진+평행이동 동시)은 더 넓은 공간에서 마지막에 시도한다.

수행 순서

- 1 전진($\text{linear.x} = +0.2$) → 후진($\text{linear.x} = -0.2$) 테스트
앞/뒤로 직진 이동
- 2 $\text{linear.y} = +0.2$ 로 평행이동 테스트 → 실제 방향(좌/우) 관찰
옆으로 이동, 방향 기록
- 3 $\text{linear.y} = -0.2$ 로 반대 방향 평행이동 테스트
반대쪽으로 이동 확인
- 4 $\text{angular.z} = +0.3$ (좌회전(turn left, CCW)) → -0.3 (우회전) 테스트
제자리에서 회전
- 5 $\text{linear.x} + \text{linear.y}$ 동시 입력으로 대각선 이동 테스트
대각선 방향으로 이동
- 6 결과 기록표 작성
6개 이동 유형 모두 기록

```

#!/usr/bin/env python3
import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class WheelTest(Node):
    def __init__(self):
        super().__init__('wheel_test')
        self.pub = self.create_publisher(Twist, '/controller/cmd_vel', 1)

    def move(self, vx=0.0, vy=0.0, wz=0.0, seconds=2.0):
        twist = Twist()
        twist.linear.x = vx
        twist.linear.y = vy
        twist.angular.z = wz
        end = time.time() + seconds
        while time.time() < end:
            self.pub.publish(twist)    # 반복 발행 - --once 유실 방지
            time.sleep(0.1)
        self.pub.publish(Twist())     # 정지도 발행으로

def main():
    rclpy.init()
    node = WheelTest()
    node.move(vx=0.2, seconds=2.0)    # 1. 전진
    time.sleep(1.0)
    node.move(vy=0.2, seconds=2.0)    # 2. 평행이동(방향은 관찰해서 기록)
    time.sleep(1.0)
    node.move(wz=0.3, seconds=2.0)    # 4. 회전
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

실습 기록

시 도	이동 유형	입력값(vx, vy, wz)	실제 관찰된 방향	판정
1	전진	0.2, 0, 0		성공/실패
2	후진	-0.2, 0, 0		성공/실패
3	평행이동 (+y)	0, 0.2, 0	좌/우 중 실제로 관찰된 방	성공/실패
4	평행이동 (-y)	0, -0.2, 0		성공/실패
5	좌/우회전	0, 0, ±0.3		성공/실패
6	대각선 이 동			성공/실패

종합 소견

linear.y의 실제 부호-방향 대응, 대각선 이동 시 속도 감소 여부 등을 자유롭게 기록

자가 점검

평가준거	평가방법	피드백 방법
6가지 이동 유형을 모두 시도했는가	실습 관찰	재시도 안내
linear.y 부호-방향 대응을 스스로 확인해 기록했는가	기록 확인	보완 요청
반복 발행 원칙을 코드에 적용했는가	코드 리뷰	즉시 교정 지도

심화 개념

- 메카넘 휠(mecanum wheel) 4개는 각각 다른 방향의 대각선 롤러를 가진다 — 네 바퀴를 같은 방향으로 돌리면 전진, 대각선 방향으로 짝을 지어 다르게 돌리면 평행이동(lateral translation)이 나온다. "네 바퀴 회전 속도의 조합"이 곧 로봇의 최종 이동 방향을 결정한다.
- 대각선 이동은 v_x 와 v_y 를 동시에 넣는 것이라 각 바퀴에 걸리는 부하가 순수 전진·평행이동보다 커서, 실제 속도가 이론값보다 느려질 수 있다.

트러블슈팅

증상	원인	해결
평행이동 시 로봇이 살짝 회전(rotation)하며 이동함	바퀴 캘리브레이션 편차 또는 무게중심 치우침	공식 매뉴얼 1.7절 속도 캘리브레이션 절차 참고
대각선 이동이 잘 안 됨	$v_x \cdot v_y$ 값이 너무 작아 마찰을 못 이김	두 값을 각각 조금씩 키워서 재시도
회전 시 제자리에서 안 돌고 이동하며 둥	angular.z만 넣었는데 이전 linear 값이 남아있음	매번 새 Twist() 객체를 생성해서 사용

개념 확인

문항	질문	답
1	메카넘 휠이 일반 바퀴와 다른 점은 무엇인가?	_____
2	대각선 이동은 어떤 값들의 조합으로 만들어지는가?	_____
3	왜 대각선 이동을 가장 마지막, 가장 넓은 공간에서 시도하라고 했는가?	_____

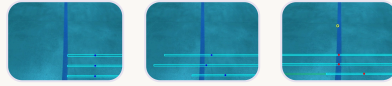
심화 과제 — $v_x \cdot v_y \cdot v_z$ 세 값을 동시에 조합해서 "회전하면서 전진"하는 나선형 경로를 만들어보세요.

🎓 메카닉 휠 역기구학(inverse kinematics)

일반적인 메카닉 휠 로봇에서, 로봇이 원하는 속도(v_x : 전진, v_y : 평행이동, ω : 회전)를 네 바퀴의 개별 회전속도로 변환하는 공식은 흔히 다음과 같은 형태를 갖는다: $v_1=v_x-v_y-\omega(l_x+l_y)$, $v_2=v_x+v_y+\omega(l_x+l_y)$, $v_3=v_x+v_y-\omega(l_x+l_y)$, $v_4=v_x-v_y+\omega(l_x+l_y)$ ($l_x \cdot l_y$ 는 바퀴 중심과 로봇 중심 사이의 가로·세로 거리). 이 로봇의 정확한 하위 컨트롤러 구현(펌웨어)이 이 공식을 그대로 쓰는지 이번 세션에서 직접 확인하지 못했다 — ROS2 계층에서는 `/controller/cmd_vel`에 $v_x \cdot v_y \cdot \omega$ 만 넘기면 그 이후 변환은 하위 컨트롤러가 담당한다.

이 공식이 왜 필요한가 — 일반 바퀴(디퍼렌셜 드라이브)는 v_y (옆으로 이동)를 만들 방법이 아예 없다. 메카닉 휠의 롤러가 45도로 붙어있어서 네 바퀴 회전속도의 조합만으로 v_y 성분을 만들어낼 수 있는 것이다. 이번 실습에서 `linear.y` 부호를 직접 실험으로 확인한 것이 바로 이 역기구학 결과를 몸으로 검증한 것이다.

06-1. 라인트레이싱 색상 캘리브레이션 — 왜 지정한 색을 못 찾았을까?



- ① 픽셀좌표로 시도 — 실패 ② 정규화 좌표로 재 시도 — 성공 ③ 3-스캔 라인 가중치 분석

실측으로 잡은 진짜 버그 —

set_target_color 서비스에 픽셀좌표 (예: 310,320)를 그대로 보냈더니①, 소스코드 안에서 $x = \text{int}(\text{point.x} * \text{이미지폭})$ 로 다시 곱해져 좌표가 화면 밖으로 튕겨나갔습니다. 화면 비율(0~1)로 바꿔 보내자② 정확히 테이프 위에 점이 찍혔습니다. 여기에 더해 로봇과 가장 가까운 스캔라인이 전체 조향 판단의 70%를 차지한다는 것도③ 코드에서 직접 확인했습니다 — 코드를 읽지 않았다면 절대 몰랐을 원인입니다.

이 레슨이 끝나면

- ✓ 색상 기반(color-based) 물체 인식이 LAB 색공간 임계값(threshold)으로 동작하는 원리를 설명할 수 있다.
- ✓ ROS2 서비스 3종(Trigger·SetBool·커스텀 SetPoint)으로 비전 앱을 켜고 끄고 설정할 수 있다.
- ✓ 소스코드를 직접 읽어 좌표계(픽셀 vs 정규화) 불일치 버그의 원인을 스스로 진단할 수 있다.
- ✓ 스캔라인 가중치 구조에서 로봇과 가까운 영역이 왜 더 중요한지 설명할 수 있다.

준비물

바닥에 붙인 색 테이프(라인)

ROSOrin Pro, 카메라 스트림 확인용 PC/브라우저

선수지식 더 보기

- [LAB 색공간 정의\(RGB와 다른 점\)](#)

L^* (밝기)· a^* (빨강-초록)· b^* (파랑-노랑) 3채널 정의 부분만 보면 됨 — MATLAB 실습 코드 부분은 건너뛰어도 무방

— MathWorks 공식 도움말(한국어)

⚠ 안전 유의사항

- 이번 레슨은 `set_running`을 켜지 않고(정지 상태) 캘리브레이션만 진행한다 — 로봇이 움직이지 않는다.
- 실제 주행 테스트는 다음 레슨(06-2)에서 별도로 다룬다.

수행 순서

- 1 **예측: 픽셀좌표(310, 320)를 그대로 보내면 어떻게 될지 먼저 예상해서 적는다**
예측 기록 완료
- 2 **enter → set_target_color (픽셀값) → 결과 화면 캡처**
사진①과 비교
- 3 **결과가 예측과 다르면, app/common.py의 ColorPicker 소스코드를 읽고 원인을 찾는다**
버그 지점 특정
- 4 **좌표를 0~1 비율로 변환해서 set_target_color 재호출 → 결과 캡처**
사진②와 비교
- 5 **결과 기록표 작성 및 소견 서술**
표 작성 완료

```
ros2 service call /line_following/enter std_srvs/srv/Trigger {}

# (실패) 픽셀좌표를 그대로 보냄 - 화면 밖으로 튕겨나감
ros2 service call /line_following/set_target_color \
  interfaces/srv/SetPoint "{data: {x: 310.0, y: 320.0, z: 0.0}}"

# (성공) 0~1 사이 정규화좌표로 변환해서 보냄
ros2 service call /line_following/set_target_color \
  interfaces/srv/SetPoint "{data: {x: 0.484, y: 0.8, z: 0.0}}"

ros2 service call /line_following/set_running std_srvs/srv/SetBool "{data:
true}"
```

실습 기록

시 도	보낸 좌표	예측(찾을까?)	실제 관찰 결과	판정
1	x=310, y=320 (픽셀값 그대로)	찾는다/못찾는다		성공/실패
2	x=0.484, y=0.8 (정규화)			성공/실패
3	본인이 고른 다른 좌표			성공/실패

종합 소견

왜 픽셀좌표는 실패하고 정규화좌표는 성공했는지, 자기 말로 설명해서 적기

자가 점검

평가준거	평가방법	피드백 방법
예측을 먼저 적고 실제 결과와 비교했는가	기록 확인	재작성 안내
소스코드를 읽고 버그 원인을 스스로 설명할 수 있는가	구술 확인	코드 재검토 지도
정규화좌표로 재시도해 성공시켰는가	실습 관찰	재시도 안내

심화 개념

- 스캔라인 가중치가 $0.7 \cdot 0.2 \cdot 0.1$ 로 로봇에 가까울수록 높은 이유 — 제어이론 관점에서 "지금 당장 눈앞의 상황"이 "저 멀리의 상황"보다 즉각적인 조향 반응에 더 중요하기 때문이다. 이런 구조를 프리뷰 제어(preview control)라 부르기도 한다.
- 같은 이유로, 근접 스캔라인 하나가 잘못 검출되면 먼 두 라인이 맞아도 전체 판단이 크게 틀어진다 — 실측으로 확인한 이번 사례가 정확히 그 경우다.

트러블슈팅

증상	원인	해결
좌표를 넣었는데 항상 화면 구석 근처의 색을 잡음	픽셀좌표를 정규화좌표 자리에 넣어 좌표를 이미지폭·높이로 나눠 값이 클램프(clamp)됨	0~1 사이 값으로 변환
정규화좌표로도 여전히 못 찾음	카메라 화이트밸런스로 인한 전체적 편향이 덜한 지점에서 재샘플링 인 색조 편향(예: 청색 캐스트)	하거나 threshold 값 조정
스캔라인 3개 중 일부만 라인을 못 찾음	라인이 화면의 그 영역(특히 로봇 근접부)까지 안 이어짐	물리적으로 라인·로봇 위치를 재정렬

개념 확인

문항	질문	답
1	set_target_color가 받는 좌표는 픽셀인가 비율인가?	_____
2	3개 스캔라인 중 가중치가 가장 높은 것은 어디에 있는 라인인가?	_____
3	근접 스캔라인 하나가 오검출되면 전체 조향에 어떤 영향을 주는가?	_____

심화 과제 — 지금 코드의 weight_sum은 항상 1.0으로 고정돼 있다. 실제로 검출에 성공한 스캔라인의 가중치만 더해서 weight_sum을 동적으로 계산하도록 고쳐보고, 근접 라인이 하나 빠졌을 때 결과가 어떻게 달라지는지 비교해보세요.

07-1. 물체 색상 추적 — 카메라로 공을 쫓아가는 로봇 만들기

이 레슨이 끝나면

- ✓ object_tracking 앱의 서비스 구조(enter·set_target_color·set_running·exit)가 06-1의 line_following과 동일한 패턴임을 확인할 수 있다.
- ✓ PID 제어기 2개(pid_yaw·pid_dist)가 각각 좌우 회전과 전후 이동을 어떻게 담당하는지 설명할 수 있다.
- ✓ set_target_color에도 0~1 정규화 좌표가 쓰인다는 06-1의 결론이 이 앱에도 그대로 적용됨을 실습으로 확인할 수 있다.
- ✓ 물체와의 정지 거리(y_stop)가 어떻게 정해지는지 소스코드로 확인하고, 실제 추적 동작과 연결지어 설명할 수 있다.

준비물

색깔 있는 공 또는 물체(라인트레이싱 테이프와 다른 색 권장)

ROSOrin Pro, 카메라 스트림 확인용 PC/브라우저

⚠ 안전 유의사항

- 이번에도 처음엔 set_running을 켜지 않고 좌표·색상 인식만 먼저 확인한다.
- 실제 추적 주행 시 로봇 전방 1.5m 이상 공간을 확보한다.
- 물체를 갑자기 가까이 들이대면 로봇이 급하게 후진할 수 있으니 천천히 접근한다.

수행 순서

- 1 **enter 호출 → 카메라 스트림에서 물체 위치 확인**
스트림 정상 표시
- 2 **set_target_color에 정규화 좌표로 물체 색 지정 → get_target_color로 인식된 RGB 확인**
RGB 값 응답 확인
- 3 **set_running=false 상태에서 물체를 좌우로 움직여 결과 화면에 원이 따라오는지만 관찰(로봇은 정지 상태)**
원이 물체를 따라 이동
- 4 **set_running=true로 전환 → 실제 추적 주행 관찰**
로봇이 물체 방향으로 회전
- 5 **물체를 가까이/멀리 움직여 pid_dist가 전후 이동을 만드는지 확인**
전진/후진 반응 확인
- 6 **결과 기록표 작성**
4가지 테스트 모두 기록

```
ros2 service call /object_tracking/enter std_srvs/srv/Trigger {}

# 0~1 정규화좌표로 목표색 지정 (06-1과 동일 원칙)
ros2 service call /object_tracking/set_target_color \
  interfaces/srv/SetPoint "{data: {x: 0.5, y: 0.4, z: 0.0}}"

ros2 service call /object_tracking/get_target_color std_srvs/srv/Trigger {}

# 좌표·색상 인식만 확인 (로봇은 정지 상태)
ros2 service call /object_tracking/set_running std_srvs/srv/SetBool "{data: false}"

# 실제 추적 주행 시작
ros2 service call /object_tracking/set_running std_srvs/srv/SetBool "{data: true}"
```

실습 기록

시도	테스트 내용	예상 반응	실제 관찰	판정
1	물체를 좌우로 이동	로봇이 물체 쪽으로 회전		성공/실패
2	물체를 가까이 이동			성공/실패
3	물체를 멀리 이동			성공/실패
4	물체를 화면 밖으로 치움(놓침)	로봇이 어떻게 반응할까		성공/실패

종합 소견

물체를 놓쳤을 때 로봇이 실제로 어떻게 반응했는지, 왜 그렇게 반응했다고 생각하는지 소스코드 근거와 함께 서술

자가 점검

평가준거	평가방법	피드백 방법
06-1과 동일한 정규화좌표 원칙을 스스로 적용했는가	실습 관찰	06-1 재확인 안내
pid_yaw·pid_dist가 각각 어떤 축을 담당하는지 설명할 수 있는가	구술 확인	코드 재검토 지도
물체를 놓쳤을 때의 동작을 관찰하고 그 이유를 소스코드로 설명했는가	기록 확인	재관찰 안내

심화 개념

- PID 세 항(P·I·D) 중 여기서는 P(비례)항만 쓰인다(Kp만 0이 아니고 I·D는 0으로 설정됨) — 목표에서 벗어난 만큼 비례해서만 반응하는 가장 단순한 제어 방식이다.
- lost_target_count 변수는 선언되고 0으로 리셋만 될 뿐, 실제로 증가시키는 코드는 소스 전체에 없다 — "물체를 몇 번 놓쳤는지 세서 재탐색하는" 로직은 이름만 있고 아직 구현되지 않은 상태임을 소스코드에서 직접 확인할 수 있다. 대신 물체를 놓치면(circle이 None) 기본값 Twist()(전부 0)가 그대로 반환되어 로봇이 자연스럽게 멈춘다.

트러블슈팅

증상	원인	해결
추적을 시작해도 로봇이 가만히 있음	set_running이 여전히 false이거나 color_picker가 아직 클리어되지 않음	get_target_color로 색이 실제로 지정됐는지 먼저 확인
물체가 가까운데 도 계속 다가감	y_stop이 처음 발견 시점의 값으로 고정되 어, 그 이후 목표 정지거리가 바뀌지 않음	set_target_color를 다시 호출해 y_stop을 재설정
회전이 너무 급 격하거나 느림	pid_yaw의 Kp(0.006)가 로봇·환경에 맞 지 않음	심화 과제처럼 Kp 값을 조정해 재 시도

개념 확인

문 항	질문	답
1	object_tracking과 line_following이 공유하는 서비스 4개는 무엇인가?	_____
2	pid_yaw와 pid_dist는 각각 어떤 축(방향)을 담당하는가?	_____
3	y_stop 값은 언제, 어떻게 정해지는가?	_____

심화 과제 — pid_yaw·pid_dist의 Kp 값(0.006, 0.002)을 바꿔가며 로봇의 반응 속도가 어떻게 달라지는지 관찰해보세요. 값을 너무 크게 하면 어떤 현상(진동·오버슈트)이 생기는지도 함께 확인해보세요.

08-1. 라이다 장애물 회피 — 가까운 쪽 반대로 피하기

이 레슨이 끝나면

- ✓ 라이다 스캔 데이터를 좌·우 절반으로 나눠 각각의 최단거리를 비교하는 장애물 회피 로직의 원리를 설명할 수 있다.
- ✓ set_running(0~3) 4가지 모드 중 이번 레슨이 다루는 모드 1(장애물 회피)을 실제로 실행하고 관찰할 수 있다.
- ✓ 라이다는 전원 인가 직후 워밍업이 필요하다는 이전 실측 사실을 알고, 정상 데이터가 나오는지 먼저 확인한 뒤 실습을 시작할 수 있다.
- ✓ threshold(회피 시작 거리)를 바꿔가며 로봇의 반응 시점이 어떻게 달라지는지 비교할 수 있다.

준비물

장애물로 쓸 상자·의자 등

ROSOrin Pro, 사방 2m 이상 여유공간

선수지식 더 보기

- [PID 제어 기초\(P·I·D 각 항의 역할과 수식\)](#)

글 상단 수식과 'P/Pi/PID' 3단 비교 부분만 보면 충분 — 뒤쪽 Ziegler-Nichols 튜닝법·C 의사코드는 심화 참고용이라 안 봐도 됨

— Sinduy's blog (CC BY 4.0, 한국어)

⚠ 안전 유의사항

- 실습 전 로봇 주변(특히 전방) 2m 이상 공간을 확보한다 — 회피 중 회전 반경이 커질 수 있다.
- 라이다 워밍업이 안 된 상태(거리값 0/NaN)에서 모드 1을 실행하면 장애물을 인식하지 못해 그대로 충돌할 수 있다 — 반드시 데이터 정상 여부를 먼저 확인한다.
- 정지 명령은 03-1·03-2와 동일하게 반복 발행으로 보낸다.

수행 순서

- 1 **ros2 topic echo /scan_raw로 거리값이 0/NaN이 아닌지 확인**
정상적인 거리값(m) 출력
- 2 **enter 호출**
서비스 응답 success
- 3 **set_running(1)로 장애물 회피 모드 시작**
로봇이 직진 시작
- 4 **로봇 왼쪽에 장애물 배치 → 반응 관찰, 이어서 오른쪽 → 양쪽 다 막고 반응 관찰**
장애물 반대 방향으로 회전
- 5 **set_param으로 threshold를 낮춰(예: 0.4) 재시도 → 반응 거리 변화 관찰**
더 가까이에서 반응
- 6 **결과 기록표 작성**
4가지 케이스 모두 기록

```
# 실행 전 라이다 워밍업 확인 (값이 0이거나 NaN이면 대기)
ros2 topic echo /scan_raw --once

ros2 service call /lidar_app/enter std_srvs/srv/Trigger {}

# 모드 1 = 장애물 회피
ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 1}"

# threshold(m), scan_angle(deg), speed(m/s) 순서
ros2 service call /lidar_app/set_param interfaces/srv/SetFloat64List \
  "{data: [0.4, 90.0, 0.2]}"
```

실습 기록

시 도	장애물 위치	예상 반응	실제 반응	판정
1	왼쪽만	오른쪽으로 회전 예상		성공/실패
2	오른쪽만			성공/실패
3	양쪽 다	180도 회전 예상		성공/실패
4	threshold 0.4로 낮춘 후 왼쪽만			성공/실패

종합 소견

threshold를 낮췄을 때 로봇이 장애물에 얼마나 더 가까이 다가간 후에 반응했는지, 체감한 차이를 자유롭게 기록

자가 점검

평가준거	평가방법	피드백 방법
실습 전 라이다 데이터 정상 여부를 확인했는가	실습 관 찰	재확인 안내
좌/우/양쪽 3가지 케이스의 회피 방향을 정확히 예측하고 확인했는가	기록 확 인	재시도 안내
threshold 파라미터의 역할을 소스코드 근거로 설명할 수 있는가	구술 확 인	코드 재검토 지 도

심화 개념

- 이 코드는 로봇 조향 방식(메카넘 vs Ackermann, MACHINE_TYPE에 'Acker' 포함 여부)에 따라 회피 로직 자체가 완전히 다른 두 갈래로 나뉜다 — Ackermann 차량은 조향각 특성상 단순 제자리 회전이 불가능해서, 회전반경(R)을 계산해 부드럽게 회피하도록 별도로 구현되어 있다.
(소스: lidar_callback의 if 'Acker' not in self.machine_type: ... else: ... 두 분기)
- 모드 2(라이다 추종)와 모드 3(라이다 경비)는 PID 제어기(pid_yaw·pid_dist)로 "가장 가까운 물체와의 각도·거리"를 유지하는 방식이라, 모드 1(단순 좌우 비교)보다 정교하지만 그만큼 구조도 복잡하다 — 이번 레슨에서는 다루지 않는다.

트러블슈팅

증상	원인	해결
장애물이 있는데 반응이 없음	라이다 워밍업 미완료로 거리값이 0/NaN	/scan_raw 값을 확인하고 12분 이상 대기 후 재시도
가까이 가도 반응이 늦음	threshold가 너무 작게 설정됨	set_param으로 threshold를 높여 재시도(1.5m 이하 범위)
회피하다가 다시 같은 장애물 쪽으로 돌	직전 회피 방향(last_act)이 번갈아 반전되도록 설계되어 있어 좁은 공간에서 진동할 수 있음	장애물 사이 폭을 넓히거나 threshold를 낮춰 재시도

개념 확인

문항	질문	답
1	set_running에 넣는 정수 1은 어떤 모드를 의미하는가?	_____
2	라이다 실습 전에 왜 /scan_raw 값을 먼저 확인해야 하는가?	_____
3	왼쪽과 오른쪽 모두 장애물이 있을 때 로봇은 어떻게 반응하는가?	_____

심화 과제 — MAX_SCAN_ANGLE(240도)을 좁혀서(예: 120도) 로봇이 더 정면의 장애물에만 반응하도록 바꿔보고, 넓은 공간에서 옆을 스쳐 지나가는 물체에 불필요하게 반응하는지 비교해보세요.

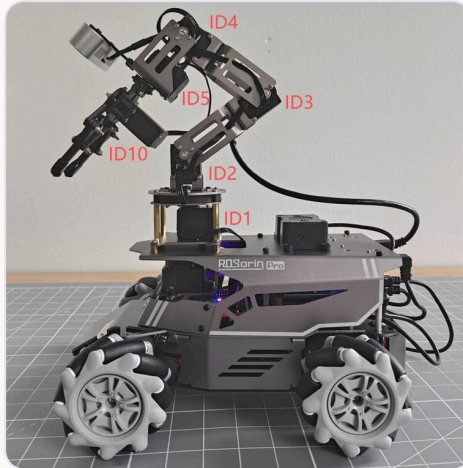
🎓 PID 제어 이론과 이 코드의 게인(gain) 값

PID 제어기는 목표값과 현재값의 차이(오차, error)에 세 가지 방식으로 반응한다 — P(비례, Proportional)는 오차 크기에 비례해서, I(적분, Integral)는 오차가 누적된 양에 비례해서, D(미분, Derivative)는 오차가 변하는 속도에 비례해서 반응한다. 최종 출력은 $output = K_p \cdot error + K_i \cdot \int error \, dt + K_d \cdot (d \, error / dt)$ 형태다.

이 lidar_controller.py의 pid_yaw는 PID(1.6, 0, 0.16), pid_dist는 PID(1.7, 0, 0.16)로 설정되어 있다 — I(적분)항은 0으로 꺼져 있고 P와 D만 쓰는 'PD 제어기'다. I항을 안 쓰는 이유로 흔히 드는 것은, 라이다 추종처럼 목표가 계속 움직이는 상황에서 I항이 오차를 계속 누적하면 과거의 오차까지 끌고 가서 오히려 진동·불안정을 키우기 쉽기 때문이다(다만 이 코드에 그런 설계 의도가 주석으로 명시되어 있지는 않다 — 일반적인 제어이론 관점의 해석이다).

D(미분)항이 0이 아니라는 것은, 오차가 빠르게 변할 때(장애물이 갑자기 가까워질 때 등) 그 변화에 미리 반응해서 진동(overshoot)을 억제하려는 의도로 흔히 해석된다.

09-1. 로봇팔 첫 동작 — 서보 (servo) 각도로 관절(joint) 움직이기



서보 ID ↔ 관절 대응(공식 매뉴얼 8장 기준) —
ID1 베이스 회전(팬) · ID2 어깨(shoulder) ·
ID3 팔꿈치(elbow) · ID4·ID5 손목(wrist) ·
ID10 그리퍼(gripper). 오늘은 이 중 **ID3(팔꿈치)**를 움직여봅니다.

이 레슨이 끝나면

- ✓ 사진에 표시된 서보 ID(1, 2, 3, 4, 5, 10)와 실제 관절 부위를 정확히 대응시켜 설명할 수 있다.
- ✓ 서보 ID·목표값(0~1000)·지속시간을 지정해 로봇팔 관절을 안전하게 움직일 수 있다.
- ✓ 실행 전 예측과 실제 결과를 비교하고, 그 차이의 이유를 논리적으로 서술할 수 있다.

준비물

없음 (소프트웨어 실습)

ROSOrin Pro, 로봇팔 주변 30cm 이상 여유공간

⚠ 안전 유의사항

- 로봇팔이 움직이는 동안 관절 사이에 손가락을 넣지 않는다.
- 최초 테스트는 중앙값(500) 근처 ± 30 이내(약 $\pm 7^\circ$)의 작은 변화로 시작한다.
- 그리퍼(서보 ID 10) 최초 테스트는 **빈손으로만** 진행한다 — 손가락 협착 위험.
- 이상 동작 시 즉시 전원 스위치로 강제 정지한다.

수행 순서

- 1 위 사진에서 ID3(팔꿈치)의 실제 위치를 눈으로 확인
움직일 관절을 정확히 가리킬 수 있음
- 2 아래 코드 작성 — ID3을 중앙값(500)에서 소량만 이동
코드 저장 완료
- 3 실행 전 예측 기록 → 실행 → 실제로 움직인 관절이 ID3(팔꿈치)가 맞는지 관찰
지정한 관절만 움직였는지 확인
- 4 예측과 실제를 비교해 실습 결과 기록표 작성
기록표 4번 항목 작성 완료

```
#!/usr/bin/env python3
import time
import rclpy
from rclpy.node import Node
from servo_controller_msgs.msg import ServosPosition
from servo_controller.bus_servo_control import set_servo_position

class ArmFirstMove(Node):
    def __init__(self):
        super().__init__('arm_first_move')
        self.joints_pub = self.create_publisher(ServosPosition,
        '/servo_controller', 1)
        time.sleep(0.5) # 퍼블리셔 연결 대기

    def move_joint(self, servo_id, position, duration=1.5):
        # position: 0~1000 (중앙 약 500), duration: 초 단위
        set_servo_position(self.joints_pub, duration, ((servo_id, position),))
        self.get_logger().info(f'서보 {servo_id} -> {position} ({duration}s)')

def main():
    rclpy.init()
    node = ArmFirstMove()
    node.move_joint(3, 520, 1.5) # ID3(팔꿈치)를 중앙에서 살짝만 이동
    time.sleep(2.0)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

실습 기록

시 도	서보ID	목표값	지속시간(s)	관찰 결과	판정
1					성공/부분/실패
2					성공/부분/실패
3					성공/부분/실패

종합 소견

ID3(팔꿈치)을 중앙값 500에서 520으로 움직이면 실제로 어느 방향으로, 얼마나 움직일 것 같습니까? 실행하기 전에 먼저 적으십시오.

자가 점검

평가준거	평가방법	피드백 방법
메시지 타입을 스스로 조회했는가	실습 관찰	재시도 안내
소각도(small angle)로 안전하게 첫 테스트를 진행했는가	실습 관찰	즉시 교정 지도
예측과 실제 결과를 비교하고 이유를 논리적으로 서술했는가	기록 확인	보완 요청

심화 개념

- 버스 서보(servo)는 하나의 시리얼 라인에 여러 서보가 데이지체인으로 연결되고, 각 서보의 ID로 개별 주소 지정이 가능한 방식이다 — 여러 조명을 하나의 리모컨으로 각각 제어하는 것과 비슷한 원리다.
- `set_servo_position()`은 여러 서보를 동시에 움직이는 것도 지원한다 — 튜플을 여러 개 넣으면(예: `((2, 500), (3, 520))`) 여러 관절(joint)이 동시에 목표 위치로 이동한다.

트러블슈팅

증상	원인	해결
코드를 실행했는데 서보가 전혀 안 움직임	서보 컨트롤 노드가 안 떠 있거나 다른 프로세스가 시리얼 포트를 점유 중	<code>ros2 node list</code> 로 서보 컨트롤러 노드 확인
서보가 목표와 다른 위치로 감	서보 자체 편차(deviation)가 누적된 상태	제조사 PC 소프트웨어로 서보 편차 보정 (공식 매뉴얼 8.1.2절)
그리퍼(gripper)가 완전히 안 닫히거나 이상한 소리가 남	기계적 한계에 도달, 서보 손상 위험	즉시 반대 방향으로 조정

개념 확인

문 항	질문	답
1	position 값 500은 대략 몇 도(240도 범위 기준)에 해당하는가?	_____
2	서보 ID를 여러 개 튜플로 넣으면 어떤 일이 일어나는가?	_____
3	그리퍼(ID10)를 처음 테스트할 때 왜 반드시 빈손이어야 하는가?	_____

심화 과제 — ID2·ID3·ID5 세 관절을 순서대로 움직이는 `move_joint` 호출을 여러 번 배치해서, "손 흔들기"처럼 보이는 동작 시퀀스를 만들어보세요.

10-1. YOLO 물체 인식 — 모델 이름 하나가 클래스 목록과 검출 방식을 함께 정한다

이 레슨이 끝나면

- ✓ yolo 앱이 /yolo/start·/yolo/stop 단 2개의 서비스만으로 제어된다는 것을 line_following·object_tracking의 4개 서비스 패턴과 비교해 설명할 수 있다.
- ✓ launch 파일의 model_name 파라미터 하나가 클래스 이름 목록과 검출 방식(detect/obb)을 동시에 결정하는 구조를 이해하고, 실제로 다른 model_name으로 실행해 차이를 관찰할 수 있다.
- ✓ conf(신뢰도 임계값) 파라미터를 바꿔가며 오탐지·미탐지가 어떻게 달라지는지 실습으로 확인할 수 있다.
- ✓ ObjectsInfo 메시지에 담기는 정보(class_name·score·box·angle)를 직접 echo해서 확인할 수 있다.

준비물

인식시킬 물체(COCO 클래스에 있는 것 — 예: 병·컵·휴대폰)

ROSOrin Pro, 카메라 스트림 확인용 PC/브라우저

선수지식 더 보기

- [신뢰도\(confidence\) 임계값과 precision/recall 트레이드오프](#)

confidence threshold를 올리고 내릴 때 precision·recall이 어떻게 반대로 움직이는지 설명하는 부분 위주 — YOLO 전용 글은 아니지만 개념은 이 레슨의 conf 파라미터에 그대로 적용됨
— velog 기술 블로그 (한국어)

⚠ 안전 유의사항

- 이번 레슨은 로봇이 움직이지 않는다 — 인식 결과만 확인한다.
- 인식률을 정상적으로 비교하려면 밝은 조명에서 진행한다.

수행 순서

- 1 **yolo_detect.launch.py**를 기본값(`model_name=yolo26n`)으로 실행 → `/yolo/start` 호출
서비스 응답 success
- 2 카메라에 COCO 클래스 물체(병·컵·휴대폰 등)를 비춰 결과 화면에 박스가 그려지는지 확인
박스·클래스명 표시
- 3 `ros2 topic echo ~/object_detect`로 `class_name·score` 값 직접 확인
메시지 정상 수신
- 4 `conf` 값을 낮춰(예: 0.3) 재실행 → 오탐지(엉뚱한 물체 검출)가 느는지 관찰
오탐지 빈도 비교
- 5 `model_name`을 `traffic`으로 바꿔 재실행 → 클래스 목록이 6개로 바뀌는지 확인
`go/right/park/red/green/crosswalk`만 표시
- 6 **결과 기록표 작성**
3가지 설정 모두 기록

```
ros2 launch example yolo_detect.launch.py conf:=0.6 model_name:=yolo26n

ros2 service call /yolo/start std_srvs/srv/Trigger {}

ros2 topic echo /yolo/object_detect

# conf를 낮춰 재실행 (오탐지 비교용)
ros2 launch example yolo_detect.launch.py conf:=0.3 model_name:=yolo26n

# 클래스 목록·검출 방식이 함께 바뀌는지 확인
ros2 launch example yolo_detect.launch.py model_name:=traffic
```

실습 기록

시도	설정(model_name / conf)	인식 시도 물체	실제 결과(class_name·score)	판정
1	yolo26n / 0.6			성공/실패
2	yolo26n / 0.3	1번과 동일한 물체		성공/실패
3	traffic / 0.6		클래스 목록이 6개로 줄었는	성공/실패

종합 소견

conf를 낮췄을 때 오탐지가 실제로 늘었는지, model_name에 따라 클래스 목록·검출 방식이 어떻게 달라졌는지 소스코드 근거와 함께 서술

자가 점검

평가준거	평가방법	피드백 방법
yolo 앱의 서비스 구조가 다른 앱과 다르다는 것을 설명할 수 있는가	구술 확인	코드 재검토 지도
model_name 변경이 클래스·검출방식에 미치는 영향을 실습으로 확인했는가	실습 관찰	재시도 안내
conf 값과 오탐지·미탐지 관계를 설명할 수 있는가	기록 확인	재관찰 안내

심화 개념

- OBB(oriented bounding box, 회전된 박스)와 일반 박스(detect)는 저장하는 좌표 형식부터 다르다 — obb는 중심좌표·너비·높이·회전각(xywhr)을, detect는 좌상단·우하단 좌표(xyxy)를 사용한다. 쓰레기 분류처럼 물체가 기울어져 놓일 수 있는 경우에만 obb를 쓰는 이유가 여기 있다. (소스: yolo_node.py의 is_obb_task 분기)
- class_name은 classes 리스트에서 class_id로 찾아오는데, 모델이 classes 리스트보다 많은 클래스로 학습되어 있었다면 "ID:숫자" 형태로 표시된다(코드의 fallback) — launch 파일의 클래스 이름 목록이 실제 모델과 정확히 일치해야 사람이 읽을 수 있는 이름이 나온다.

트러블슈팅

증상	원인	해결
박스는 그려지는데 이름이 ID:3 같이 나옴	launch의 classes 리스트가 실제 모델 학습 클래스 수·순서와 안 맞음	model_name에 맞는 올바른 클래스 리스트로 재실행
인식이 하나도 안 됨	conf 임계값이 너무 높거나 물체가 클래스 목록에 없음	conf를 낮춰보고, COCO 80개 클래스에 있는 물체로 재시도
~/yolo_start_detect 서비스가 안 보임	/yolo/start를 아직 호출하지 않았거나 카메라 프레임이 한 장도 처리되지 않음	/yolo/start 호출 후 카메라 앞에 물체를 비춰 최소 1프레임 처리시키기

개념 확인

문항	질문	답
1	yolo 앱은 몇 개의 서비스로 제어되는가? line_following과 몇 개 다른가?	_____
2	model_name에 'garbage'가 들어가면 어떤 두 가지가 함께 바뀌는가?	_____
3	class_name이 "ID:7"처럼 숫자로만 나온다면 무엇을 의심해봐야 하는가?	_____

심화 과제 — FPS 계산 코드($\text{self.fps} = \text{self.fps} \times 0.9 + \text{current_fps} \times 0.1$)는 지수이동평균(EMA) 방식입니다. 계수를 0.9에서 0.5로 바꾸면 화면에 표시되는 FPS 값이 더 빨리/느리게 반응하는지 실습으로 확인해보고, 그 이유를 설명해보세요.

🎓 지수이동평균(EMA)과 FPS 계산

`self.fps = self.fps*0.9 + current_fps*0.1`은 지수이동평균(Exponential Moving Average, EMA)의 전형적인 형태다 — 일반화하면 $EMA_t = \alpha \cdot EMA_{(t-1)} + (1-\alpha) \cdot x_t$ 이고, 여기서 $\alpha=0.9$ 다. 매 프레임 새로 측정한 순간 FPS(x_t)는 화면에 표시되는 값에 10%만 반영되고, 나머지 90%는 이전까지 누적된 값을 그대로 이어받는다.

이게 왜 필요한가 — 순간 FPS는 프레임마다 크게 흔들린다(카메라 읽기·추론 시간이 매번 조금씩 다르므로). α 를 1에 가깝게 두면(예: 0.9) 화면에 찍히는 숫자가 부드럽게 변하지만, 실제 성능이 갑자기 나빠져도 반영되기까지 시간이 걸린다(반응 지연, lag). 반대로 α 를 낮추면(예: 0.5) 실제 변화에 빠르게 반응하지만 프레임마다 숫자가 들쭉날쭉해 보인다 — 이게 advancedTask에서 0.9와 0.5를 비교해보라고 한 이유다.

이 구조는 신호처리의 1차 저역통과 필터(first-order low-pass filter)와 수학적으로 동일하다 — 여기서는 FPS 표시에만 쓰였지만, 센서 노이즈를 부드럽게 하고 싶은 어떤 값(예: 배터리 전압, 거리 측정값)에도 같은 공식을 그대로 적용할 수 있다. α 값 하나가 "얼마나 부드럽게, 얼마나 빠르게 반응할지"라는 트레이드오프를 전부 결정한다는 것이 이 공식의 핵심이다.

11-1. 음성으로 로봇 움직이기 — 웨이크워드부터 '이리와'까지

이 레슨이 끝나면

- ✓ 벤더 소스코드에서 "웨이크워드가 무엇인가"를 확인할 때는 로그 문자열이 아니라 실제 동작을 결정하는 파라미터 기본값을 봐야 한다는 것을 실제 사례로 이해한다.
- ✓ 기본 음성 명령 4종(전진·후진·좌회전·우회전)이 각각 어떤 Twist 값과 지속시간으로 변환되는지 설명할 수 있다.
- ✓ "이리와" 명령이 라이다 각도 정보로 화자 방향을 향해 회전한 뒤 라이다 추종 모드로 전환되는 2단계 동작임을 설명할 수 있다.
- ✓ Ackermann 조향 로봇에서는 "이리와" 명령 자체가 지원되지 않는다는 것을 소스코드 조건문에서 확인할 수 있다.

준비물

없음 (음성 실습)

ROSOrin Pro(마이크 정상 연결 상태), 조용한 실내 공간

⚠️ 안전 유의사항

- 음성 명령 중에도 로봇은 실제로 움직인다 — 사방 1m 이상 공간을 확보한다.
- "이리와" 테스트 시 라이다 워밍업이 안 되어 있으면(08-1 참고) 추종 단계가 오작동할 수 있다.

수행 순서

- 1 웨이크워드("샤오환샤오환") 발화 → 웨이크업 성공 신호(효과음) 확인
효과음 재생
- 2 15초 이내에 "전진" 발화 → 로봇이 2초간 전진 후 자동 정지하는지 관찰
2초 이동 후 정지
- 3 같은 방식으로 "후진"·"좌회전"·"우회전" 각각 테스트
각 명령대로 동작
- 4 로봇에서 1~2m 떨어져 "이리와" 발화 → 로봇이 화자 방향으로 회전한 뒤 다가오는지 관찰
회전 후 접근
- 5 결과 기록표 작성
5가지 명령 모두 기록

동작 확인용 토픽 구독 (음성 명령은 서비스 호출이 아니라 발화로 트리거됨)

```
ros2 topic echo /asr_node/voice_words
```

```
ros2 topic echo /awake_node/angle
```

실습 기록

시도	발화 명령	예상 동작	실제 동작	판정
1	웨이크워드(샤오환샤오환)	효과음 재생		성공/실패
2	전진			성공/실패
3	좌회전			성공/실패
4	이리와	회전 후 접근		성공/실패

종합 소견

자가 점검

평가준거	평가방법	피드백 방법
실제 기능 기본값(파라미터)과 로그 문자열을 구분해서 설명할 수 있는가	구술 확인	소스코드 재검토 지도
4가지 기본 명령과 "이리와"의 동작 차이를 설명할 수 있는가	실습 관찰	재시도 안내
Ackermann 로봇에서 "이리와"가 왜 빠지는지 소스코드 근거로 설명했는가	기록 확인	코드 재검토 지도

심화 개념

- "이리와" 명령의 회전각 계산은 감지된 각도가 90도보다 크고 270도보다 작은지에 따라 회전 방향·잔여 회전량 계산식이 달라진다 — 마이크 배열이 보고하는 각도 기준(0~360도, 로봇 정면 기준)을 이해해야 이 분기가 왜 이렇게 나뉘는지 알 수 있다.
- count 변수가 10에 도달하면(연속 10회 정지 상태) start_follow를 자동으로 끈다 — "가까이다 왔다고 판단되면 추종을 스스로 멈추는" 타임아웃 성격의 안전장치다.

트러블슈팅

증상	원인	해결
웨이크업이 안 됨	웨이크워드 발음이 정확하지 않거나 마이크 하드웨어 문제	정확한 발음(샤오환샤오환)으로 재시도, 마이크 연결 상태 확인
전진 등은 되는데 이리와만 안 됨	MACHINE_TYPE이 ROSOrin_Acker인 경우 이 기능 자체가 코드에서 제외됨	로봇 사양 확인(메카넘 계열 인지)
이리와 회전은 하는데 그 후 다가오지 않음	라이다 데이터가 비정상(0/NaN)이라 추종 로직이 작동하지 않음	08-1의 라이다 워밍업 절차 먼저 수행

개념 확인

문 항	질문	답
1	이 프로젝트 소스코드에서 로그의 "hello hiwonder"와 실제 파라미터 값 "샤오환샤오환"이 다른 이유는 무엇이라고 추정되는가?	_____
2	음성 명령 하나를 말하면 로봇은 얼마나 오래 움직이는가?	_____
3	"이리와" 명령은 왜 Ackermann 조향 로봇에서 지원되지 않는가?	_____

심화 과제 — time_stamp 기반 "2초 후 자동 정지" 로직을 실제 코드에서 찾아, 이 값을 1초로 바꾸면 체감상 로봇 반응이 어떻게 달라지는지 실습해보세요.

12-1. LLM 기반 자연어 이동 제어 — 문장 한 줄이 좌표·시간 값으로 바뀌는 과정

이 레슨이 끝나면

- ✓ 사용자의 자연어 명령이 시스템 프롬프트를 거쳐 [x, y, z, t] 형식의 이동 명령 리스트로 변환되는 전체 흐름을 설명할 수 있다.
- ✓ ASR_LANGUAGE 환경변수가 'Chinese'와 그 외(영어) 두 값만 구분한다는 것, 그리고 이 프로젝트의 한글화 작업이 분기 조건이 아니라 프롬프트 "내용"만 번역했다는 것을 소스코드로 직접 확인할 수 있다.
- ✓ action 리스트의 마지막 항목이 항상 [0.0, 0.0, 0.0, 0.0]이어야 하는 이유(정지 보장)와 그것이 강제되지 않는다는 점을 함께 설명할 수 있다.
- ✓ 웨이크업 중 실행 중인 동작을 끊는 interrupt 메커니즘의 동작을 설명할 수 있다.

준비물

없음 (음성+LLM 실습)

ROSOrin Pro(온라인 LLM API 연결 필요), 인터넷 연결

선수지식 더 보기

- [eval\(\)의 보안 위험과 안전한 대안](#)

'위험' 섹션과 '안전한 사용법·대안' 섹션 위주로 — eval() 기본 사용법 설명(도입부)은 이미 코드로 봤으니 건너뛰어도 됨

— Leapcell 기술 블로그 (한국어)

⚠ 안전 유의사항

- LLM이 예상 밖의 action 값을 생성할 수 있으므로 사방 1.5m 이상 공간을 확보한다.
- 처음 실습할 때는 간단한 한 동작짜리 명령("앞으로 2초 이동해줘")부터 시작한다.

수행 순서

- 1 웨이크워드 발화 → LLM 노드가 응답 대기 상태인지 확인
웨이크업 신호 확인
- 2 "앞으로 2초 이동한 다음 왼쪽으로 1초 회전해줘"처럼 프롬프트 예시와 비슷한 문장으로 명령
지시대로 이동
- 3 로봇의 실제 동작과 음성 응답(response) 문구를 함께 기록
응답·동작 모두 기록
- 4 동작 실행 중 다시 웨이크워드를 불러 interrupt가 실제로 동작을 멈추는지 확인
즉시 정지
- 5 "공중부양해줘"처럼 존재하지 않는 동작을 명령해 action이 빈 리스트로 나오고 응답만 오는지 확인
동작 없이 응답만
- 6 결과 기록표 작성
4가지 케이스 모두 기록

```
# 프롬프트 요구사항 핵심 발체 (llm_control_move.py)
# "action": [[x1,y1,z1,t1], [x2,y2,z2,t2], ..., [0.0,0.0,0.0,0.0]]
# x,y ∈ [-1.0, 1.0] (선속도, m/s) / z ∈ [-1.0, 1.0] (각속도, rad/s) / t (지속시간, s)

# action -> Twist 변환 (process() 내부, 실제 소스)
for i in action_list:
    msg = Twist()
    msg.linear.x = float(i[0])
    msg.linear.y = float(i[1])
    msg.angular.z = float(i[2])
    self.mecanum_pub.publish(msg)
    time.sleep(i[3])
    if self.interrupt:
        self.interrupt = False
        self.mecanum_pub.publish(Twist())
        break
```

실습 기록

시 도	발화 명령	예상 동작(action 개수)	실제 동작·응답	판정
1	단일 동작 (예: 앞으로 2초 이동)	1개 + 정지		성공/실패
2	복합 동작 (전진 + 회전)			성공/실패
3	동작 중 웨이크워드로 interrupt	즉시 정지		성공/실패
4	존재하지 않는 동작(공중부양)	action=[]		성공/실패

종합 소견

ASR_LANGUAGE 실제 값 확인 결과와, 그에 따라 어느 언어의 프롬프트가 실제로 쓰이고 있었는지 소스코드 근거와 함께 서술

자가 점검

평가준거	평가방법	피드백 방법
자연어 명령 → action 리스트 → Twist 변환 흐름을 설명할 수 있는가	구술 확인	코드 재검토 지도
ASR_LANGUAGE 분기와 번역 작업의 관계를 정확히 구분해 설명했는가	기록 확인	소스코드 재검토 지도
interrupt 메커니즘을 실습으로 확인했는가	실습 관찰	재시도 안내

심화 개념

- 이 코드는 `eval()`로 LLM 응답 문자열을 직접 파싱한다(`eval(self.llm_result[...])`) — LLM이 요청한 JSON 형식대로 정확히 답하지 않으면 이 부분에서 예외가 날 수 있다. 실무에서는 임의 코드 실행 위험 때문에 `eval` 대신 `json.loads`를 쓰는 것이 더 안전하다.
- 이 프롬프트는 "간결하고(5~10자) 재치 있는" 응답 문구를 매번 새로 생성하도록 요청한다 — 같은 명령이라도 매번 다른 음성 응답이 나올 수 있는 것은 버그가 아니라 의도된 프롬프트 설계다.

트러블슈팅

증상	원인	해결
명령했는데 로봇이 안 움직이고 응답만 함	LLM이 "동작을 찾을 수 없다"고 판단해 <code>action</code> 을 빈 리스트로 반환함	프롬프트 예시와 비슷한 표현으로 재시도
동작이 끝났는데도 로봇이 계속 움직임	LLM이 마지막에 <code>[0,0,0,0]</code> 을 안 붙였을 가능성(프롬프트 요청일 뿐 강제 아님)	수동으로 정지 명령 발행, 필요 시 프롬프트 예시를 더 명확히 보강
<code>eval</code> 관련 에러가 로그에 남음	LLM 응답이 요청한 JSON 형식과 다르게 옴	프롬프트를 더 구체화하거나 같은 명령을 다시 시도

개념 확인

문항	질문	답
1	<code>action</code> 리스트 각 항목의 4개 숫자는 각각 무엇을 의미하는가?	_____
2	<code>ASR_LANGUAGE</code> 가 'Korean'으로 설정되어 있다면 이 코드는 어느 프롬프트를 쓰는가?	_____
3	<code>interrupt</code> 는 언제, 어떻게 실행 중인 동작을 멈추는가?	_____

심화 과제 — 이 프롬프트에 "속도는 항상 0.5 이하로 제한"이라는 안전 규칙을 한 줄 추가해보고, LLM이 실제로 그 제한을 지키는 응답을 여러 번 생성하는지 테스트해서 일관성을 확인해보세요.

🎓 프롬프트 엔지니어링과 eval()의 보안 문제

이 프롬프트가 형식 예시(few-shot example)를 2개 넣어둔 것은 우연이 아니다 — LLM에게 규칙을 말로만 설명하는 것보다 실제 입출력 예시를 보여주는 것이 훨씬 안정적으로 원하는 형식을 재현하게 만든다는 것이 프롬프트 엔지니어링의 기본 원칙 중 하나다.

코드가 LLM 응답을 eval()로 파싱하는 것은 보안 관점에서 위험한 패턴이다 — eval()은 문자열을 파이썬 코드로 그대로 실행하기 때문에, 만약 LLM이 (오작동이든 프롬프트 주입이든) JSON이 아닌 임의의 파이썬 코드를 응답에 포함시키면 그대로 실행될 수 있다. 정석적인 방법은 json.loads()로 파싱해서 JSON이 아니면 예외로 처리하는 것이다 — 실행 가능한 코드가 아니라 순수 데이터로만 취급하기 때문에 이런 위험이 없다.

13-1. 쓰레기 분류 로봇 — YOLO 인식부터 잡기·분류까지 하나로 잇기

이 레슨이 끝나면

- ✓ waste_classification 앱이 자체 검출 로직 없이 yolo 앱의 서비스(/yolo/start, /yolo/stop)를 그대로 호출해 재사용한다는 것을 설명할 수 있다.
- ✓ 물체가 "충분히 멈춰 있다"고 판단하는 기준(연속 10프레임, 유클리드 거리 5mm 이내)을 설명하고 왜 이런 안정성 확인이 필요한지 이해한다.
- ✓ 4가지 쓰레기 카테고리(음식물·유해·재활용·일반)에 속한 실제 클래스 목록을 소스코드에서 확인하고, 각 카테고리가 어느 좌표로 이송되는지 설명할 수 있다.
- ✓ 실제 팔 자세(관절 각도) 계산은 별도의 kinematics 서비스가 담당하고, 이 앱은 "어디로 가야 하는지" 목표 좌표만 넘긴다는 역할 분담 구조를 이해한다.

준비물

WASTE_CLASSES에 해당하는 실물(예: 플라스틱병·장난감 등으로 대체 가능)

ROSOrin Pro(카메라·로봇팔·YOLO 'garbage' 모델 설치 상태)

⚠ 안전 유의사항

- 로봇팔이 실제로 움직이므로 팔 동작 반경 내에 손이나 물건을 두지 않는다.
- 처음 실습 시 target으로 한 카테고리만 지정해서(예: recyclable_waste만) 테스트 범위를 좁힌다.

수행 순서

- 1 **enter 호출 → 카메라·YOLO 서브스크립션 활성화 확인**
서비스 응답 success
- 2 **set_target으로 목표 카테고리 지정(예: ["recyclable_waste"])**
target_list 로그 확인
- 3 **enable_transport(true) 호출 → yolo/start가 내부적으로 호출되는지 로그로 확인**
YOLO 시작 로그 확인
- 4 **목표 카테고리 물체를 카메라 앞에 정지 상태로 놓기 → 잠시 대기 후 잡기 동작이 시작되는지 관찰**
잡기 동작 시작
- 5 **잡은 물체가 해당 카테고리의 목적지 좌표로 이송되는지 관찰**
올바른 위치에 놓음
- 6 **결과 기록표 작성**
4가지 케이스 모두 기록

```
ros2 service call /waste_classification/enter std_srvs/srv/Trigger {}
```

```
ros2 service call /waste_classification/set_target \
  interfaces/srv/SetStringList "{data: ['recyclable_waste']}"
```

```
ros2 service call /waste_classification/enable_transport \
  std_srvs/srv/SetBool "{data: true}"
```

실습 기록

시 도	목표 카테고리	놓은 물체	실제 분류 결과	판정
1	recyclable_waste			성공/실패
2	food_waste			성공/실패
3	목표 리스트에 없는 물체	무시되는지 확인		성공/실패
4	물체를 계속 움직이며 놓기	count_move로 스킵		성공/실패

종합 소견

물체가 안정된 것으로 판단되기까지 체감 대기시간이 얼마나 걸렸는지, 목표 리스트에 없는 물체는 실제로 무시됐는지 서술

자가 점검

평가준거	평가방법	피드백 방법
waste_classification이 yolo 앱을 재사용하는 구조를 설명할 수 있는가	구술 확인	코드 재검토 지도
안정성 판단 기준(count_still/count_move)을 실습으로 확인했는가	실습 관찰	재시도 안내
좌표 계산과 관절 각도 계산의 역할 분담을 설명할 수 있는가	기록 확인	코드 재검토 지도

심화 개념

- `go_home()` 함수는 카테고리별로 복귀 대기시간(t)이 다르게 설정되어 있다 (recyclable_waste=2.0초, hazardous_waste=1.7초, food_waste=1.4초, residual_waste=1.0초) — `place_position`까지의 거리·회전량이 카테고리마다 달라서 실측으로 맞춘 값으로 추정되나, 정확한 산출 근거는 소스코드 주석에 남아있지 않아 확인되지 않는다.
- `target_list`가 비게 되면(목표를 다 처리하면) `target_list_temp`에서 자동으로 복사해 다시 채운다 — 한 카테고리를 다 치우면 라운드가 자동으로 리셋되어 계속 순찰하듯 동작하는 구조다.

트러블슈팅

증상	원인	해결
물체를 봐도 안 잡음	<code>class_name</code> 이 <code>target_list</code> 에 없거나 <code>enable_transport</code> 가 꺼져있음	<code>set_target</code> 으로 정확한 카테고리·클래스명 지정 후 <code>enable_transport(true)</code> 재호출
물체가 흔들리는 동안 계속 안 잡힘	<code>count_still</code> 기준(연속 10프레임, 5mm 이내)을 못 채움 — 의도된 안정성 확인 동작	물체를 완전히 고정한 후 대기
잡았는데 엉뚱한 통에 놓음	<code>WASTE_CLASSES</code> 카테고리 매핑 또는 <code>place_position</code> 좌표가 실제 분리수거함 배치와 안 맞음	<code>place_position</code> 좌표를 실제 배치에 맞게 재조정

개념 확인

문항	질문	답
1	<code>waste_classification</code> 앱은 물체 검출을 직접 하는가, 다른 앱의 서비스를 빌려쓰는가?	_____
2	<code>count_still</code> 이 10을 넘어야 잡기를 시작하는 이유는 무엇인가?	_____
3	로봇팔이 목표 좌표까지 어떻게 굽혀야 하는지는 이 앱이 계산하는가, 다른 서비스가 계산하는가?	_____

심화 과제 — `WASTE_CLASSES`에 새 카테고리(예: 'special_waste')를 하나 추가하고 `place_position`에 도 좌표를 추가해보세요. YOLO 모델 자체는 재학습하지 않으므로, 기존 클래스 중 하나를 새 카테고리에 재

배정하는 방식으로 실습해보세요.

14-1. 색상+AprilTag 물체 분류 — 두 가지 인식 방식을 하나로 합치기

이 레슨이 끝나면

- ✓ object_sorting 앱이 색상 인식(LAB blob)과 AprilTag 인식(dt_apriltags) 두 가지 방법을 하나의 목표 리스트로 합쳐 처리한다는 것을 설명할 수 있다.
- ✓ 13-1(쓰레기 분류)과 동일한 안정성 판정(5mm·10프레임) 패턴이 이 앱에서도 독립적으로 재 구현되어 있음을 확인한다.
- ✓ 벤더 소스코드에서 실제로 동작하지 않는 코드(go_home())의 오타·도달 불가능한 분기)를 스스로 찾아내고, 왜 그런데도 프로그램이 멈추지 않는지 설명할 수 있다.
- ✓ place_position이 색상 3종과 AprilTag 3종, 총 6개 목적지로 구성된다는 것을 소스코드에서 확인할 수 있다.

준비물

빨강·초록·파랑 물체, AprilTag(tag36h11, ID 1~3) 인쇄물

ROSOrin Pro

선수지식 더 보기

- [동차좌표\(homogeneous coordinates\) 정의와 카메라 투영 연결](#)

글 도입부(동차좌표 정의·수학적 표현)와 후반부 '카메라 projection과의 관계' 부분 위주로 — 중간의 세부 파생 공식은 건너뛰어도 무방

— gaussian37 블로그 (Computer Vision 전문 블로그, 한국어)

⚠ 안전 유의사항

- 로봇팔이 실제로 움직이므로 팔 동작 반경 내에 손이나 물건을 두지 않는다.
- 처음 실습 시 목표를 한 종류(색상 하나 또는 태그 하나)만 지정해서 테스트 범위를 좁힌다.

수행 순서

- 1 **enter → set_target으로 목표(색상 또는 태그) 지정 → enable_sorting(true)**
서비스 응답 success
- 2 **색상 물체 하나 놓기 → 분류·이송 관찰**
해당 색 목적지로 이송
- 3 **AprilTag 물체 하나 놓기 → 같은 방식으로 분류되는지 관찰**
해당 태그 목적지로 이송
- 4 **소스코드에서 go_home()을 직접 찾아 "bule" 오타와 if/elif 구조를 확인 → t 값이 실제로 쓰이는 곳이 있는지 함수 전체를 다시 읽기**
버그 지점 특정
- 5 **결과 기록표 작성**
3가지 케이스 모두 기록

```
ros2 service call /object_sorting/enter std_srvs/srv/Trigger {}

ros2 service call /object_sorting/set_target \
  interfaces/srv/SetStringBool "{data_str: 'red', data_bool: true}"

ros2 service call /object_sorting/enable_sorting \
  std_srvs/srv/SetBool "{data: true}"
```

실습 기록

시 도	목표 종류	놓은 물체	실제 결과	판정
1	색상(예: red)			성공/실패
2	AprilTag(예: tag1)			성공/실패
3	목표로 지정 안 한 색/ 태그	무시되는지 확인		성공/실패
4	go_home() 코드 리뷰 결과	t가 실제로 쓰이는 줄0		찾음/못찾음

종합 소견

go_home()의 버그가 실제 로봇 동작에는 왜 영향이 없는지, 소스코드 근거와 함께 자기 말로 설명

자가 점검

평가준거	평가방법	피드백 방법
색상·AprilTag 두 인식 방식이 하나로 합쳐지는 구조를 설명할 수 있는가	구술 확인	코드 재검토 지도
13-1과 동일한 안정성 판정 패턴을 스스로 알아봤는가	기록 확인	13-1 재비교 안내
go_home()의 버그를 스스로 찾고 그 영향(없음)을 설명했는가	코드 리뷰 결과 확인	재검토 지도

심화 개념

- AprilTag는 격자무늬 흑백 마커로, 카메라가 코너 4개의 픽셀좌표만 찾으면 마커의 3D 자세(위치+회전)까지 계산할 수 있다 — 색상 인식과 달리 조명 변화에 강하고 회전각도까지 정확히 알 수 있는 것이 장점이다.
- 이번 go_home() 사례처럼, "코드가 존재한다고 해서 그 코드가 실제로 의도한 대로 실행된다는 보장은 없다." 벤더 소스코드를 신뢰하는 것과, 코드의 흐름을 끝까지 따라가며 검증하는 습관은 서로 다른 이야기라는 것을 이번 실습으로 직접 확인했다.

트러블슈팅

증상	원인	해결
색상 인식이 안 됨	lab_config.yaml의 색상 임계값이 조명 환경과 안 맞음	조명을 밝게 하거나 임계값 재조정
AprilTag 인식이 안 됨	태그 크기(tag_size=0.025m) 또는 family(tag36h11)가 실제 인쇄물과 안 맞음	정확한 tag36h11 계열 태그를 지정된 크기로 재인쇄
목표로 지정 안 한 물체도 잡음	set_target으로 target_labels가 의도대로 설정되지 않음	set_target 호출 결과를 로그로 재확인

개념 확인

문항	질문	답
1	object_sorting은 YOLO를 쓰는가, 다른 방식으로 물체를 찾는가?	
2	go_home()의 elif 체인이 왜 "bule"이 아닌 목표에 대해서는 절대 실행되지 않는가?	
3	이 버그가 실제 로봇 동작에는 문제를 일으키지 않는 이유는 무엇인가?	

심화 과제 — go_home()의 오타("bule"→"blue")와 elif 구조를 직접 고쳐서 순수 if 3개로 바꿔보세요. 고친 버전과 원본이 실제 로봇 동작에서 다르게 보이는지(t가 원래 안 쓰이므로 겉보기엔 차이가 없을 수 있음) 확인해보고 그 이유를 설명해보세요.

🎓 픽셀 좌표에서 로봇 좌표로 — 카메라 좌표 변환의 원리

카메라가 보는 것은 2D 픽셀 좌표(x, y)뿐이지만, 로봇팔은 3D 실세계 좌표(x, y, z , 단위: m)로 움직여야 한다. 이 변환에는 카메라 내부 파라미터(intrinsic — 초점거리·주점 등, camera_info 토픽에서 얻음)와 외부 파라미터(extrinsic — 카메라가 로봇 기준으로 어디에 어떤 각도로 달려있는지)가 모두 필요하다.

동차좌표(homogeneous coordinates)는 3D 좌표에 1을 하나 더 붙여($[x, y, z, 1]$) 회전과 이동(translation)을 하나의 행렬 곱셈으로 한 번에 표현하는 방법이다 — 이 소스코드의 projection_matrix가 바로 회전(extristic[1])과 이동(extristic[0])을 하나로 합친 동차변환행렬이다.

이런 계산을 매 프레임 새로 하지 않고 calibration_*.yaml 파일에 미리 보정된 값(offset·scale)을 저장해두고 재사용하는 이유는, 카메라·로봇팔의 위치 관계가 조립 후에는 거의 바뀌지 않는 고정값이기 때문이다 — 한 번 정밀하게 캘리브레이션해두면 매번 다시 계산할 필요가 없다.

15-1. 손 제스처로 로봇 제어하기 — 궤적의 기울기로 방향을 읽는 법

이 레슨이 끝나면

- ✓ `hand_trajectory`와 `hand_gesture_control` 두 노드가 커스텀 토픽(Points)으로 손끝 궤적 데이터를 주고받는 2단계 파이프라인 구조를 설명할 수 있다.
- ✓ "하나"(검지만 펴기) 제스처로 궤적 기록을 시작하고 "다섯"(손바닥 펴기) 제스처로 종료·발행하는 상태 전이(state machine) 흐름을 설명할 수 있다.
- ✓ 기록된 궤적에 직선을 피팅(fitLine)해서 그 기울기(각도)만으로 상·하·좌·우 스와이프를 판별하는 원리를 이해한다.
- ✓ 이 프로젝트에서 로봇팔을 움직이는 3가지 방법(개별 서보 지정·좌표 기반 역기구학·미리 녹화된 액션그룹 재생)을 비교해서 설명할 수 있다.

준비물

없음 (제스처 실습)

ROSOrin Pro(카메라·로봇팔), 손 제스처를 취할 사용자

선수지식 더 보기

- [MediaPipe Hands 21개 손 랜드마크](#)

21개 랜드마크가 정규화 좌표([0,1] 범위)로 표현된다는 정의 부분만 보면 충분
— 푸르고 개발 블로그 (한국어, 원문은 Google MediaPipe 공식문서)

⚠ 안전 유의사항

- 팔이 미리 녹화된 동작을 재생하므로 팔 동작 반경 내에 손이나 물건을 두지 않는다.
- 조명이 밝아야 MediaPipe 손 검출 정확도가 높아진다.

수행 순서

- 1 **hand_trajectory 노드의 /start 호출 → 카메라에 손을 비춰 랜드마크가 그려지는지 확인**
손 골격 표시 확인
- 2 **"하나"(검지만 펴기) 제스처를 잠깐 유지 → 궤적 기록(TRACKING) 시작 확인**
궤적 선 그려짐
- 3 **손을 위→아래로 움직인 뒤 "다섯"(손바닥 펴기)으로 종료 → 팔이 정해진 동작을 재생하는지 관찰**
hand_control_pick 재생
- 4 **같은 방식으로 좌→우 궤적 시도 → 다른 동작이 재생되는지 확인**
hand_control_place 재생
- 5 **결과 기록표 작성**
4가지 케이스 모두 기록

```
ros2 service call /hand_trajectory/start std_srvs/srv/Trigger {}
```

```
ros2 topic echo /hand_trajectory/points
```

```
ros2 service call /hand_trajectory/stop std_srvs/srv/Trigger {}
```

실습 기록

시 도	제스처 시퀀스	예상 동작	실제 동작	판정
1	하나→위에서 아래로→다섯			성공/실패
2	하나→왼쪽에서 오른쪽으로→다섯			성공/실패
3	너무 빨리 그은 궤적(점이 적음)			성공/실패
4	"다섯"을 계속 안 보여준 경우	TRACKING이 계속 유		성공/실패

종합 소견

직선 피팅 각도로 방향을 나누는 방식의 한계(대각선으로 그으면 어떻게 되는지 등)를 직접 시도해 보고 서술

자가 점검

평가준거	평가방법	피드백 방법
두 노드 사이의 데이터 흐름(Points 토픽)을 설명할 수 있는가	구술 확인	코드 재검토 지도
상태 전이(제스처→기록 시작/종료)를 실습으로 확인했는가	실습 관찰	재시도 안내
로봇팔 제어 3가지 방식의 차이를 비교해 설명할 수 있는가	기록 확인	09-1·13-1 재비교 안내

심화 개념

- 이미 다룬 09-1·13-1·14-1과 비교하면, 이 프로젝트는 로봇팔을 움직이는 서로 다른 세 가지 방식을 모두 갖고 있다 — ① 개별 서보 값을 직접 지정(09-1), ② 목표 좌표만 주면 kinematics 서비스가 역기구학을 계산(13-1·14-1), ③ 미리 녹화한 동작 시퀀스를 이름으로 재생(이번 레슨의 ActionGroupController). 정밀 제어가 필요하다면 ①·②를, 반복되는 정형 동작은 ③을 쓰는 것이 합리적이다.
- 궤적에 점이 하나만 있어도 fitLine은 동작하지만 의미 있는 기울기가 나오지 않는다 — hand_trajectory_node가 이전 점과 5px 미만 차이인 새 점은 기록에서 아예 제외하는 것 (`distance(last_point, index_finger_tip) < 5`)도 이런 노이즈를 줄이기 위한 장치로 보인다.

트러블슈팅

증상	원인	해결
"하나" 제스처를 해도 궤적 기록이 시작 안 됨	5프레임 연속 유지 조건을 못 채우거나 조명이 어두워 손 검출 실패	제스처를 더 오래 유지하고 조명을 밝게 조정
궤적은 그렸는데 팔이 반응 안 함	"다섯" 제스처로 정상 종료되지 않아 Points가 발행되지 않음	손바닥을 확실히 펴서 "다섯"으로 인식시키기
엉뚱한 방향으로 판정됨	궤적이 대각선에 가까워 60~90도/0~30도 경계에 걸침	더 수평 또는 수직에 가깝게 크게 그어서 재시도

개념 확인

문항	질문	답
1	"하나" 제스처와 "다섯" 제스처는 각각 어떤 역할을 하는가?	_____
2	궤적의 방향은 어떤 방법으로 판별하는가?	_____
3	이 로봇에서 로봇팔을 움직이는 세 가지 방식은 각각 무엇인가?	_____

심화 과제 — `h_gesture()`의 임계값(`thr_angle=65`, `thr_angle_thumb=53`, `thr_angle_s=49`)을 조금씩 바꿔가며, 자신의 손 크기·카메라 각도에서 제스처 인식이 더 안정적으로 되는 값을 찾아보세요.

16-1. 자율주행 통합 — 차선·신호 등·표지판을 하나의 판단으로 묶기

이 레슨이 끝나면

- ✓ self_driving 앱이 06-1(라인트레이싱)의 3-스캔라인 가중치 알고리즘과 10-1(YOLO)의 'traffic' 모델(6개 클래스: go·right·park·red·green·crosswalk)을 그대로 재사용해서 조합한다는 것을 확인할 수 있다.
- ✓ 횡단보도까지의 거리를 y축 픽셀 값으로 근사하고, 여러 하드코딩된 임계값 (160·225·300·365)을 적용해 감속·정지·주차 시점을 판단하는 방식을 설명할 수 있다.
- ✓ 빨간불이 "가까이" 있을 때만(박스 면적>700) 정지하고 너무 멀면 무시하는 실제 로직을 확인하고 그 이유를 추론할 수 있다.
- ✓ 우회전 표지 감지 시 이진화 이미지에 "가상 유도선"을 그려 넣어 차선 추종 알고리즘이 자연스럽게 회전하도록 유도하는 트릭을 이해한다.

준비물

신호등·표지판 인쇄물(go·right·park·red·green·crosswalk), 라인트레이싱용 노란 선 트랙

ROSOrin Pro

⚠ 안전 유의사항

- 로봇이 자율로 주행하므로 트랙 주변 충분한 공간을 확보하고 즉시 정지시킬 수 있도록 대기한다.
- 처음엔 문제가 생기면 바로 set_running(false)을 호출할 수 있게 명령을 미리 준비해둔다.

수행 순서

- 1 enter → set_running(true)로 자율주행 시작, 노란 선 트랙을 정상 주행하는지 확인
차선 추종 주행
- 2 빨간불 표지판을 트랙 위 먼 지점에 놓기 → 로봇이 무시하고 계속 가는지 관찰(면적 < 700 예상)
정지하지 않음
- 3 빨간불 표지판을 가까이(로봇 진행 방향 바로 앞) 놓기 → 정지하는지 관찰(면적 > 700 예상)
정지함
- 4 횡단보도 표지 + park 표지를 함께 배치 → 감속 후 주차 동작이 실행되는지 관찰
주차 동작 실행
- 5 우회전 표지를 곡선 구간 앞에 배치 → 자연스럽게 우회전하는지 관찰
우회전 성공
- 6 결과 기록표 작성
5가지 케이스 모두 기록

```
ros2 service call /self_driving/enter std_srvs/srv/Trigger {}
```

```
ros2 service call /self_driving/set_running std_srvs/srv/SetBool "{data: true}"
```

문제 발생 시 즉시 정지

```
ros2 service call /self_driving/set_running std_srvs/srv/SetBool "{data: false}"
```

실습 기록

시 도	상황	예상 동작	실제 동작	판정
1	먼 빨간불	무시하고 주행		성공/실패
2	가까운 빨간불	정지		성공/실패
3	초록불	감속 후 통과		성공/실패
4	횡단보도+park 표지			성공/실패
5	우회전 표지			성공/실패

종합 소견

빨간불 정지 거리(면적 700 기준)가 체감상 몇 cm 정도였는지, 우회전이 부드러웠는지 서술

자가 점검

평가준거	평가방법	피드백 방법
06-1·10-1의 알고리즘이 이 앱에 재사용된다는 것을 설명 할 수 있는가	구술 확인	06-1·10-1 재비교 안내
빨간불 거리 조건(area>700)을 실습으로 확인했는가	실습 관찰	재시도 안내
우회전 가상선 트릭의 원리를 소스코드 근거로 설명할 수 있는가	코드 리뷰 결과 확인	재검토 지도

심화 개념

- 이 lane_detect의 3-스캔라인 가중치($0.7 \cdot 0.2 \cdot 0.1$)는 06-1의 line_following과 동일한 설계 원칙을 공유한다 — 같은 프로젝트 안에서 "라인 추종"이라는 핵심 알고리즘 하나를 여러 앱(line_following, self_driving)이 재사용하고 있음을 확인할 수 있다.
- park_action()은 로봇 조향 방식(Mecanum·Ackermann·기타)마다 완전히 다른 3가지 주차 동작 시퀀스를 갖고 있다 — 메카넘은 옆으로 미끄러져 들어가고, Ackermann은 전후진을 반복하는 3점 주차(3-point turn)에 가까운 동작을 한다.

트러블슈팅

증상	원인	해결
빨간불을 봐도 안 멈춤	신호등 박스 면적이 700 이하 — 아직 너무 멀리 있는 것으로 판단됨	표지판을 더 가까이 배치하거나 크게 인쇄
횡단보도에서 감속을 아예 안 함	count_crosswalk가 3에 도달하기 전에 표지가 화면에서 사라짐(3프레임 연속 조건)	표지판이 화면에 안정적으로 계속 보이도록 각도·거리 조정
우회전이 너무 급격하거나 안 됨	가상선의 y 위치(y-60)가 트랙 폭·속도와 안 맞음	심화 과제처럼 관련 상수를 조정해 재시도

개념 확인

문항	질문	답
1	self_driving이 차선·표지판을 인식할 때 새로운 인식 알고리즘을 만드는가, 기존 것을 재사용하는가?	_____
2	빨간불을 봤는데도 로봇이 멈추지 않는 경우는 언제인가?	_____
3	우회전 표지를 감지했을 때 로봇은 실제로 어떤 트릭으로 회전을 유도하는가?	_____

심화 과제 — 빨간불 정지 판단 기준(area>700)을 다른 값으로 바꿔서, 로봇이 더 멀리서 또는 더 가까워서 정지하도록 튜닝해보고 트랙과의 적정 거리를 찾아보세요.

17-1. LLM 함수 호출(function calling)

— AI가 도구를 고르고 로봇이 실행한다

이 레슨이 끝나면

- ✓ 12-1(고정 JSON 형식 강제)과 이 앱의 함수 호출(function calling) 방식의 차이를 설명할 수 있다 — 이번엔 LLM이 "어떤 도구를, 어떤 인자로 부를지"를 스스로 선택한다.
- ✓ tools 리스트에 등록된 12개 이상의 함수(위치 이동·라인트레이싱·물체추적·직접이동 등)가 실제로는 이 노드의 일반 메서드일 뿐이며, 실제 호출은 함수 이름을 문자열로 비교하는 if/elif로 연결된다는 것을 소스코드에서 확인할 수 있다.
- ✓ move_to_location 도구가 enum으로 이동 가능한 장소 목록을 제한해서, LLM이 존재하지 않는 장소로 이동하라고 하는 것을 애초에 막는 설계를 이해한다.
- ✓ 벤더 소스 자체의 영어·한글 설명 불일치(90도 회전 시간 8초 vs 4초)를 스스로 찾아내고, 이런 불일치가 왜 위험한지(LLM이 잘못된 숫자를 참고할 수 있음) 설명할 수 있다.

준비물

없음 (음성+LLM 실습)

ROSOrin Pro(온라인 LLM API 연결 필요), 인터넷 연결

선수지식 더 보기

- [LLM function calling\(도구 호출\) 개념](#)

개념 정의와 '문제상황 → 해결책' 도입부 위주 — 뒤쪽 상세 구현 코드는 이 레슨의 실제 소스 발췌로 대체 가능하니 건너뛰어도 됨

— velog 기술 블로그 (한국어)

⚠ 안전 유의사항

- LLM이 예상 밖의 동작을 생성할 수 있으므로 사방 1.5m 이상 공간을 확보한다.
- 이동 관련 도구(move_to_location, robot_move_control)를 테스트할 때는 특히 주의 깊게 관찰한다.

수행 순서

- 1 "지금 뭐가 보여?" 같은 질문 → `describe_current_view` 도구가 호출되는지 로그로 확인
해당 도구 호출 로그
- 2 "빨간 선을 따라가줘" → `line_following(color='red')` 도구가 호출되는지 확인
`line_following` 호출
- 3 "침실로 이동해줘"처럼 존재하는 장소 이동 요청 → `move_to_location` 호출 확인
`move_to_location` 호출
- 4 "은하계로 이동해줘"처럼 존재하지 않는 장소 요청 → `enum`에 없어서 거부/다른 응답이 오는지 관찰
이동 시도 없음
- 5 결과 기록표 작성
4가지 케이스 모두 기록

```
# tools 스키마 등록 예시 (실제 소스 발췌)
{
  "type": "function",
  "function": {
    "name": "move_to_location",
    "description": "로봇을 지정된 위치로 이동시킵니다",
    "parameters": {
      "type": "object",
      "properties": {
        "destination": {
          "type": "string",
          "enum": ["Study_room(서재)", "Bedroom(침실)", "Kitchen(주방)"]
        }
      }
    },
    "required": ["destination"]
  }
}

# 실행 디스패치 (실제 소스 발췌, if/elif 문자열 비교)
elif tool_name == 'move_to_location':
    destination = arguments['destination']
    res = self.move_to_location(destination)
```

실습 기록

시 도	요청 문장	예상 호출 도구	실제 로그	판정
1	화면 설명 요청	describe_current_vie		성공/실패
2	라인트레이싱 요 청	line_following		성공/실패
3	존재하는 장소 이 동	move_to_location		성공/실패
4	존재하지 않는 장 소 이동	호출 안 됨 예상		성공/실패

종합 소견

12-1 방식과 비교했을 때 이 방식이 더 안정적으로 느껴졌는지, enum 제약이 실제로 잘못된 이동을 막아줬는지 서술

자가 점검

평가준거	평가방법	피드백 방법
12-1과 이 앱의 LLM 제어 방식 차이를 설명할 수 있는가	구술 확인	12-1 재비교 안내
enum 제약의 안전장치 역할을 실습으로 확인했는가	실습 관찰	재시도 안내
영어·한글 설명 불일치를 스스로 찾고 그 위험성을 설명했는가	기록 확인	소스코드 재검토 지도

심화 개념

- 이 구조는 "LLM이 직접 로봇을 조종하는 코드를 생성"하는 것이 아니라 "이미 검증된 함수 중 하나를 선택해서 호출"하는 것이다 — 12-1에서 다룬 `eval()`의 보안 위험과 비교하면 훨씬 안전한 설계다. LLM이 아무리 이상한 응답을 해도, `tool_name`이 등록된 함수 이름과 정확히 일치하지 않으면 아무 동작도 실행되지 않는다.
- `get_object_pixel`(원래는 `object`의 오타로 보임)처럼 함수 이름에 오타가 있어도, `tools` 스키마와 실제 `elif` 분기 양쪽에 똑같은 오타로 등록만 되어 있으면 시스템은 정상 동작한다 — 함수 "이름"은 프로그래머와 LLM 사이의 임의의 약속일 뿐, 철자가 맞고 틀리고는 동작에 영향이 없다는 것을 보여주는 사례다.

트러블슈팅

증상	원인	해결
엉뚱한 도구가 호출됨	요청 문장이 모호해서 LLM이 다른 도구를 선택함	더 구체적인 문장으로 재요청
존재하지 않는 장소로 보내달라 했는데 반응이 없음	enum 제약으로 LLM이 애초에 그 값을 선택 못함(의도된 안전장치)	정상 동작 — <code>get_available_locations</code> 로 실제 가능한 장소 확인
회전 시간이 설명과 다름	벤더 소스 자체의 영어·한글 설명 불일치(8초 vs 4초)	실측으로 실제 값을 확인해 기록

개념 확인

문항	질문	답
1	12-1의 방식과 이 앱의 function calling 방식은 무엇이 다른가?	_____
2	<code>move_to_location</code> 의 enum은 어떤 문제를 방지하기 위한 것인가?	_____
3	<code>robot_move_control</code> 설명문의 영어와 한글 중 어느 쪽이 실제로 맞는지 어떻게 확인할 수 있는가?	_____

심화 과제 — 실제 로봇으로 $\text{angular.z}=0.5\text{rad/s}$ 로 90도($\pi/2$ 라디안) 회전시켜 걸리는 시간을 스톱워치로 재보고, 소스코드의 영어(8초)·한글(4초) 중 어느 쪽에 더 가까운지 확인해보세요.

18-1. LLM 색상 선택 라인트레이싱 — 세 번째 방식: 함수처럼 생긴 문자열을 정규식으로 읽기

이 레슨이 끝나면

- ✓ 이 프로젝트가 LLM 출력을 로봇 동작으로 바꾸는 세 번째 방식(문자열로 표현된 가짜 함수 호출을 정규식으로 파싱)을 12-1(숫자 배열)·17-1(구조화된 tool calling)과 비교해서 설명할 수 있다.
- ✓ 06-1의 line_following 서비스(enter·set_color·set_running)가 이번에도 그대로 재사용된다는 것을 확인할 수 있다.
- ✓ 정규식 `line_following\(['^']+')`이 LLM이 만든 `"line_following('red')"` 같은 문자열에서 실제 색상값만 뽑아내는 원리를 이해한다.
- ✓ 재웨이크업(다시 부르기) 시 진행 중이던 동작이 중단되고 새 명령을 받을 준비 상태로 돌아가는 흐름을 설명할 수 있다.

준비물

색깔 선(라인트레이싱용, 최소 검은색)

ROSOrin Pro(온라인 LLM API 연결 필요)

⚠ 안전 유의사항

- LLM이 예상 밖의 동작을 생성할 수 있으므로 사방 1.5m 이상 공간을 확보한다.
- 라인트레이싱 실습이므로 06-1의 안전 수칙(저속 시작 등)을 함께 지킨다.

수행 순서

- 1 "검은 선을 따라가줘" 발화 → line_following 서비스가 호출되고 실제 추종 주행 시작
확인
라인 추종 시작
- 2 주행 중 다시 웨이크워드로 불러서 다른 명령 → 진행 중이던 추종이 멈추고 새 명령을 받
는지 확인
동작 중단·재대기
- 3 line_following과 무관한 엉뚱한 요청(예: "노래 불러줘") → action이 빈 배열로 나오
고 응답만 오는지 확인
동작 없이 응답만
- 4 결과 기록표 작성
3가지 케이스 모두 기록

```
# 프롬프트가 요구하는 출력 형식(실제 소스 발체)
{"action": ["line_following('black')"], "response": "알겠습니다"}

# 색상값 추출 (실제 소스 발체)
pattern = r"line_following\(['(^[^']+)+'\]"
match = re.search(pattern, action_string)
if match:
    color = match.group(1) # 예: 'black', 'red'
```

실습 기록

시 도	발화 명령	예상 동작	실제 동작	판정
1	검은 선 추종 요청	라인 추종 시작		성공/실패
2	주행 중 재호출로 중 단			성공/실패
3	무관한 요청(노래 불 러줘 등)	동작 없음		성공/실패

종합 소견

12-1·17-1과 비교했을 때 이 방식의 장단점을 자기 말로 정리해서 서술

자가 점검

평가준거	평가방법	피드백 방법
세 가지 LLM 출력 처리 방식(12-1·17-1·이번 레슨)의 차이를 설명할 수 있는가	구술 확인	세 레슨 재비교 안내
06-1 서비스가 재사용되는 것을 확인했는가	실습 관찰	06-1 재확인 안내
재웨이크업 시 동작 중단 흐름을 실습으로 확인했는가	실습 관찰	재시도 안내

심화 개념

- 이제 이 프로젝트 안에서 LLM 출력을 로봇 동작으로 바꾸는 3가지 방식을 전부 확인했다 — ① 12-1의 순수 숫자 배열(가장 단순, eval() 위험 있음) ② 17-1의 구조화된 tool calling(가장 안전·확장하기 좋음, 이중 관리 필요) ③ 이번 레슨의 "함수처럼 생긴 문자열 + 정규식"(그 중간, LLM이 비교적 자유롭게 생성하지만 정규식이 걸러줌). 실무에서 새 LLM 로봇 기능을 설계한다면 ②를 기본으로 삼는 것이 유지보수에 유리하다.
- 정규식 파싱 방식의 한계 — 이 프롬프트의 "동작 함수 라이브러리"에는 `line_following('black')` 예시 하나만 적혀 있지만 실제 정규식은 어떤 색상 문자열이든 다 통과시킨다. 즉 프롬프트 문서와 실제 코드가 허용하는 범위가 정확히 일치하지 않는다(문서는 좁게, 코드는 넓게) — 실제 어떤 입력까지 허용되는지는 코드를 직접 읽어야만 확실히 알 수 있다는 교훈을 준다.

트러블슈팅

증상	원인	해결
요청해도 라인 트레이싱이 시작 안 됨	LLM이 <code>line_following('색상')</code> 형식과 다르게 응답해 정규식이 매치 안 됨	"~선을 따라가줘" 형태로 더 명확하게 재요청
색을 지정했는데 다른 색을 따라감	이전 색상 설정이 초기화되지 않고 남아있음	<code>line_following/enter</code> 를 다시 호출해 상태 초기화
재호출해도 멈추지 않음	<code>wakeup_callback</code> 조건(<code>llm_result</code> 존재 또는 <code>not self.stop</code>)에 안 걸림	동작이 진행 중일 때 정확히 재호출했는지 타이밍 확인

개념 확인

문 항	질문	답
1	이 앱이 LLM 출력에서 색상값을 뽑아내는 방법은 무엇인가?	_____
2	12-1·17-1·이번 레슨의 LLM 출력 처리 방식을 한 문장씩으로 요약해보라.	_____
3	이 파일의 이름(llm_visual_patrol.py)과 실제 클래스·노드 이름이 다른 것은 무엇을 시사하는가?	_____

심화 과제 — 프롬프트의 "동작 함수 라이브러리"에 line_following 외에 다른 동작(예: 존재하지 않는 stop_line_following())을 추가해보고, 실제로 그 함수가 호출되도록 process()의 정규식·처리 로직도 함께 확장해보세요.

19-1. 낙상 감지 로봇 — 사람이 넘어졌는지 픽셀 좌표 평균으로 판단하기

이 레슨이 끝나면

- ✓ MediaPipe Pose로 얻은 신체 랜드마크의 평균 y좌표 하나만으로 "서 있는지·누워 있는지"를 판단하는 단순한 휴리스틱을 이해한다.
- ✓ 3프레임 다수결(2/3 이상) 판정 방식이 순간적인 오검출을 걸러내는 원리를 설명할 수 있다.
- ✓ 낙상이 감지되면 부저 경고와 로봇의 전후 흔들림 동작이 동시에 실행되는 스레드 구조를 확인할 수 있다.
- ✓ 이 예제는 cv2.imshow로 결과를 표시해서 ROS 토픽으로 스트리밍되지 않는다는 것, 즉 웹 브라우저로는 결과를 볼 수 없고 로봇에 모니터가 연결되어 있어야 한다는 실무적 제약을 이해한다.

준비물

안전한 매트(낙상 연기용)

ROSOrin Pro(모니터 또는 VNC 연결 필요 — cv2.imshow 기반이라 브라우저 스트리밍 불가)

선수지식 더 보기

- [MediaPipe Pose 신체 랜드마크\(33개 키폰트\) 구조](#)
33개 키폰트가 {x, y, z, visibility}로 표현된다는 정의 부분 위주 — 뒤쪽 스쿼트 판정 응용 사례는 참고만 해도 무방
— velog 기술 블로그 (한국어)

⚠ 안전 유의사항

- 실습자가 직접 "쓰러지는 연기"를 할 때는 안전한 매트 위에서 진행한다.
- 로봇이 경고 시 전후로 흔들리므로 로봇 주변 여유 공간을 확보한다.

수행 순서

- 1 노드 실행 → 모니터에 카메라 화면과 신체 랜드마크가 그려지는지 확인
랜드마크 표시
- 2 정상적으로 서 있는 상태 유지 → 부저·흔들림이 발생하지 않는지 확인
경고 없음
- 3 매트 위에 눕는 동작(낙상 흉내) → 3프레임 판정 후 부저+흔들림이 시작되는지 관찰
경고 발생
- 4 다시 일어나기 → 짧은 확인음 한 번과 함께 경고가 종료되는지 관찰
경고 종료
- 5 결과 기록표 작성
3가지 케이스 모두 기록

```
# 낙상 판정 핵심 로직 (실제 소스 발췌)
def height_cal(landmarks):
    y = [p[1] for p in landmarks]
    return sum(y) / len(y)

# image_proc() 안에서
h = height_cal(landmarks)
if h > image.shape[:2][0] - 120: # 이미지 높이 - 120px
    self.fall_down_count.append(1)
else:
    self.fall_down_count.append(0)

if len(self.fall_down_count) == 3:
    count = sum(self.fall_down_count)
    self.fall_down_count = []
    if count > 1: # 3프레임 중 2번 이상 낙상 판정
        # 부저 경고 + 흔들림 동작 동시 시작
```

실습 기록

시 도	상황	예상 반응	실제 반응	판정
1	정상 서있기	경고 없음		성공/실패
2	눕기(낙상 흉 내)	부저+흔들림 시작		성공/실패
3	다시 일어나 기	확인음 1회 후 종료		성공/실패

종합 소견

임계값(120px)이 실제 카메라 설치 높이·각도에서 적절했는지, 오탐지가 있었는지 서술

자가 점검

평가준거	평가방법	피드백 방법
평균 y좌표 기반 판정 원리를 설명할 수 있는가	구술 확인	코드 재검토 지도
3프레임 다수결의 목적을 설명할 수 있는가	구술 확인	06-1·13-1 재비교 안내
낙상·회복 각각의 경고 동작을 실습으로 확인했는가	실습 관찰	재시도 안내

심화 개념

- 이 판정 방식은 카메라 각도·거리에 의존하는 상대적 휴리스틱이다 — 카메라를 낮게 설치하면서 서 있는 사람도 화면 아래쪽에 몰려 오탐지가 늘 수 있다. 실제 안전 제품이라면 랜드마크 y좌표 평균보다 더 견고한 특징(예: 신체 세로·가로 비율, 시간에 따른 자세 변화 속도 등)을 함께 쓰는 것이 일반적이다.
- cv2.imshow를 쓰는 예제(이 낙상 감지, qrcode_detector 등)는 로봇에 모니터가 몰려있거나 VNC로 데스크톱에 접속했을 때만 결과를 볼 수 있다 — 이 프로젝트의 대다수 앱 (line_following, object_tracking 등)처럼 result_image를 ROS 토픽으로 발행해 web_video_server로 스트리밍하도록 바꾸면 브라우저에서도 확인할 수 있게 개선할 수 있다.

트러블슈팅

증상	원인	해결
누워도 낙상으로 인식 안 됨	카메라 각도상 몸 전체가 화면에 안 들어옴	camera_up 액션이 정상 실행됐는지, 카메라 각도·거리 재조정
서 있는데도 계속 오탐지됨	이미지 높이-120px 기준이 카메라 설치 높이와 안 맞음	임계값(120)을 실제 환경에 맞게 조정
브라우저로 결과 화면이 안 보임	이 예제는 cv2.imshow 기반이라 ROS 토픽으로 발행되지 않음(설계상 제약)	로봇에 직접 모니터 연결 또는 VNC 접속

개념 확인

문항	질문	답
1	낙상 여부를 판단하는 핵심 값은 무엇이며 어떻게 계산되는가?	_____
2	왜 한 프레임이 아니라 3프레임을 모아서 판정하는가?	_____
3	이 예제의 결과 화면을 왜 웹 브라우저로 볼 수 없는가?	_____

심화 과제 — `image.shape[:-2][0]` - 120의 "120"을 다른 값으로 바꿔가며, 자신의 카메라 설치 각도·높이에서 오탐지가 가장 적은 값을 찾아보세요.

20-1. 사람 따라가기 — 뎀스카메라로 실제 거리를 재는 추적

이 레슨이 끝나면

- ✓ 07-1(픽셀 y좌표로 거리를 추정하는 방식)과 이 앱(뎀스카메라로 실제 mm 단위 거리를 재는 방식)의 차이를 비교해서 설명할 수 있다.
- ✓ YOLO의 COCO 기본 모델(10-1에서 배운 80개 클래스)에서 'person' 클래스 하나만 추적 대상으로 삼는 것을 확인할 수 있다.
- ✓ 목표 지점 주변 작은 사각형(ROI) 안의 깊이값 평균을 구하고, 목표거리(150cm) 근처는 그대로 유지하는 데드존(dead zone) 설계를 이해한다.
- ✓ 사람의 바운딩박스에서 "중심점"이 아니라 상단에서 일정 비율 내려온 지점을 추적 기준으로 삼는 이유를 추론할 수 있다.

준비물

없음 (사람이 직접 추적 대상)

ROSOrin Pro(뎀스카메라), 모니터 또는 VNC 연결

⚠ 안전 유의사항

- 추적 대상(사람)과 로봇 사이 항상 1.5m 이상 여유를 두고 처음 테스트한다.
- cv2.imshow 기반이라(19-1과 동일한 제약) 모니터·VNC 연결이 필요하다.

수행 순서

- 1 노드 실행 → 카메라 앞에 서서 중심점(노란 점)이 몸에 표시되는지 확인
중심점 표시
- 2 앞뒤로 움직이며 로봇이 150cm 거리를 유지하려 하는지 관찰
거리 유지
- 3 좌우로 움직이며 로봇이 회전으로 따라오는지 관찰
회전 추적
- 4 여러 명이 동시에 화면에 있을 때 어떤 사람을 따라가는지 관찰
대상 확인
- 5 결과 기록표 작성
3가지 케이스 모두 기록

```
# 거리·각도 제어 핵심 로직 (실제 소스 발췌)
roi = self.depth_frame[h_1:h_2, w_1:w_2] # 기준점 주변 10x10 픽셀
distances = roi[np.logical_and(roi > 0, roi < 40000)]
distance = int(np.mean(distances) / 10) # mm -> cm

if abs(distance - 150) < 15:
    distance = 150 # 목표거리(150cm) 근처는 데드존
    self.pid_d.update(distance - 150)
    self.linear_x = -common.set_range(self.pid_d.output, -0.4, 0.4)

if abs(self.center[0] - w/2) < 30:
    self.center[0] = w/2 # 화면 중앙 근처도 데드존
    self.pid_angular.update(self.center[0] - w/2)
```

실습 기록

시 도	상황	예상 동작	실제 동작	판정
1	전후 거리 유지	150cm 유지		성공/실패
2	좌우 회전 추적			성공/실패
3	다수 인원 등장	어느 한 명을 따라감		성공/실패

종합 소견

07-1과 비교했을 때 이 앱의 추적이 더 안정적으로 느껴졌는지, 다수 인원 상황에서 어떤 사람을 따라갔는지 서술

자가 점검

평가준거	평가방법	피드백 방법
07-1과 이 앱의 거리 측정 방식 차이를 설명할 수 있는가	구술 확인	07-1 재비교 안내
데드존의 목적을 실습으로 확인했는가	실습 관찰	재시도 안내
다수 인원 상황에서의 동작을 관찰하고 이유를 설명했는가	기록 확인	코드 재검토 지도

심화 개념

- `get_object_callback()`은 여러 사람이 감지되면 마지막으로 처리된 `person` 객체의 좌표로 `self.center`를 계속 덮어쓴다 — 반복문에 `break`가 없어서 "가장 먼저 발견한 사람"이 아니라 "목록에서 가장 나중에 나온 사람"을 따라가게 될 수 있다. 여러 명이 있는 환경에서는 예측하지 못한 대상을 따라갈 수 있다는 뜻이다.
- 07-1의 픽셀 기반 거리 추정과 이 앱의 뎀스카메라 기반 거리 측정을 비교하면, 뎀스카메라 방식이 조영·물체 크기에 덜 민감한 대신 유효 거리 범위(40000mm 이내)와 뎀스카메라 자체의 오차 범위에 의존한다는 트레이드오프가 있다.

트러블슈팅

증상	원인	해결
사람을 따라가 지 않음	YOLO가 'person'으로 인식 못함(조명·자세 문제) 또는 데스값이 유효범위 밖	밝은 곳에서 정면으로 서서 재시도
거리는 맞는데 회전이 안 됨	center[0]이 화면 중앙 데드존($\pm 30\text{px}$) 안 에 계속 있음	더 크게 좌우로 이동해서 재시도
영동한 사람을 따라감	여러 명이 감지될 때 마지막 처리된 사람이 채택되는 구조	심화 개념 참고 — 의도된 동작이 아니라 코드 구조상 나타나는 현상

개념 확인

문항	질문	답
1	이 앱이 사람과의 거리를 재는 방법은 07-1과 어떻게 다른가?	
2	데드존은 왜 필요한가?	
3	여러 사람이 동시에 화면에 있으면 로봇은 누구를 따라가는가?	

심화 과제 — `get_object_callback()`을 고쳐서 "가장 가까운 사람"이나 "가장 큰 바운딩박스를 가진 사람"을 따라가도록 로직을 바꿔보고 어떻게 달라지는지 실험해보세요.

🎓 데드존(dead zone) 설계 — 왜 목표값 근처에서는 일부러 반응하지 않는가

센서 값은 완벽하게 고정되지 않는다 — 사람이 가만히 서 있어도 웹스카메라가 재는 거리(distance)나 바운딩박스 중심(center)은 프레임마다 몇 mm·몇 px씩 미세하게 흔들린다. 만약 컨트롤러가 이 미세한 흔들림에도 매번 반응한다면, 로봇은 목표에 도달한 뒤에도 계속 미세하게 전진·후진·회전을 반복하며 떠는 현상(hunting, 헌팅)이 생긴다 — 에너지 낭비는 물론 기구부 마모, 불안정해 보이는 동작으로 이어진다.

이 코드는 목표값 주변에 **허용 구간(tolerance band)**을 두어 이 문제를 막는다 — if $\text{abs}(\text{distance}-150)<15$: $\text{distance}=150$ 은 실제 거리가 135~165cm 사이 어디든 강제로 150(=오차 0)으로 취급해버린다. PID 컨트롤러 입장에서는 오차가 정확히 0이니 아무 것도 하지 않는다. 화면 중앙 판정($\text{abs}(\text{center}[0]-w/2)<30$)도 동일한 원리다.

데드존 폭은 트레이드오프다 — 너무 좁으면(예: $\pm 2\text{cm}$) 여전히 자잘한 떨림이 남고, 너무 넓으면(예: $\pm 50\text{cm}$) 사람이 실제로 30cm 가까워져도 로봇이 반응하지 않아 "둔하다"고 느껴진다. 이 코드가 택한 $\pm 15\text{cm}$ · $\pm 30\text{px}$ 이라는 구체적인 수치는 실측으로 튜닝된 값으로 보이며, 정답이 하나로 정해진 공식은 없고 "실제 하드웨어의 노이즈 크기"에 맞춰 결정해야 하는 설계 파라미터다. 가정용 보일러 온도조절기가 목표온도 ± 0.5 도 안에서는 켜고 끄기를 반복하지 않는 것도 같은 원리(히스테리시스, hysteresis)다.

21-1. 지정한 사람만 따라 움직이기 — 몸짓 인식 + 옷 색상으로 대상 고정하기

이 레슨이 끝나면

- ✓ "허리에 손 얹기" 자세가 이동 명령이 아니라 "지금부터 이 사람의 옷 색을 기준으로 삼겠다"는 캘리브레이션 트리거로 쓰인다는 것을 이해한다.
- ✓ 06-1·07-1에서 이미 배운 ColorPicker가 이번엔 라인이나 물체가 아니라 "사람 몸통 중심 점"에 적용되어, 신체 인식과 색상 인식을 결합해 특정 인물을 구분하는 원리를 설명할 수 있다.
- ✓ 팔·다리를 들어올리는 자세가 관절 사이 거리 비율로 판별되고, 4프레임 다수결로 확정되는 과정을 설명할 수 있다.
- ✓ Ackermann 조향 로봇의 회전 동작이 이 앱에서는 Twist(cmd_vel)가 아니라 개별 바퀴 모터에 직접 회전수(RPS)를 지정하는 더 낮은 수준의 제어로 구현되어 있다는 것을 소스코드에서 확인할 수 있다.

준비물

색이 뚜렷한 상의(캘리브레이션용)

ROSOrin Pro(웹스카메라), 모니터 또는 VNC 연결

⚠ 안전 유의사항

- 로봇이 몸짓에 따라 움직이므로 사방 1.5m 이상 공간을 확보한다.
- Ackermann 로봇에서는 회전 동작이 8초간 지속되므로 충분한 공간에서 테스트한다.

수행 순서

- 1 노드 실행 → 카메라 앞에서 허리에 손 얹기 자세 유지 → 캘리브레이션 완료 알림(부저) 확인
부저 알림
- 2 다른 색 옷을 입은 사람이 끼어들었을 때 제어가 안 되는지 확인(can_control=False 예상)
제어 불가 확인
- 3 캘리브레이션된 사람이 한쪽 팔을 들어올리기 → 로봇이 해당 방향으로 이동하는지 관찰
이동 반응
- 4 팔을 내리기 → 로봇이 정지하는지 확인
정지
- 5 결과 기록표 작성
4가지 케이스 모두 기록

```
# 색상 캘리브레이션 + 인물 판정 핵심 로직 (실제 소스 발췌)
if self.lock_color is None and self.current_color is not None and self.calibrating:
    self.lock_color = self.current_color[1] # 옷 색상 저장
    self.buzzer_warn() # 캘리브레이션 완료 알림

if self.lock_color is not None and self.current_color is not None:
    res = get_dif(list(self.lock_color), list(self.current_color[1]))
    self.can_control = (res < 50) # 색상 차이가 작으면 같은 사람으로 판단
```

실습 기록

시 도	자세·상황	예상 동작	실제 동작	판정
1	캘리브레이션	부저 알림		성공/실패
2	다른 색 옷 사람 개입	제어 안 됨		성공/실패
3	왼팔 들기			성공/실패
4	팔 내리기	정지		성공/실패

종합 소견

옷 색상 기반 인물 구분이 실제로 얼마나 안정적이었는지, Ackermann 로봇이라면 회전 동작이 다른 레슨과 어떻게 다르게 느껴졌는지 서술

자가 점검

평가준거	평가방법	피드백 방법
캘리브레이션 자세와 이동 자세의 역할 차이를 설명할 수 있는가	구술 확인	코드 재검토 지도
옷 색상 기반 인물 구분 원리를 06-1·07-1과 연결해 설명할 수 있는가	구술 확인	06-1·07-1 재비교 안내
Ackermann 로봇의 저수준 모터 제어 방식을 다른 레슨과 비교해 설명했는가	기록 확인	코드 재검토 지도

심화 개념

- 옷 색상 기반 인물 구분은 값싸고 계산이 가벼운 대신, 비슷한 색 옷을 입은 다른 사람이 나타나면 쉽게 혼동될 수 있는 한계가 있다(get_dif 임계값 50이 상당히 관대함). 실제 사람 재식별(re-identification)에는 이보다 정교한 특징(옷 무늬·얼굴·걸음걸이 등)을 함께 쓰는 것이 일반적이다.
- Twist(cmd_vel) 대신 MotorsState로 개별 바퀴를 직접 제어하는 것은 더 정밀한 제어가 가능하지만, cmd_vel 위에서 동작하는 다른 안전장치(반복 발행 관례 등)를 우회한다는 뜻이기도 하다 — 왜 이 특정 동작만 더 낮은 계층으로 내려갔는지는 소스 주석에 명시되어 있지 않다.

트러블슈팅

증상	원인	해결
캘리브레이션이 안 됨	허리에 손 얹기 자세가 각도 조건과 안 맞음	카메라를 정면으로 보고 자세를 더 명확하게 취하기
캘리브레이션했는데 제어가 안 됨	옷 색상이 배경·조명과 비슷해 get_dif 값이 임계값(50)을 넘음	더 뚜렷한 색 옷으로 재시도
팔을 들어도 반응이 늦음	4프레임 다수결 조건(3/4 이상)을 못 채움	자세를 좀 더 오래 유지

개념 확인

문항	질문	답
1	"허리에 손 얹기" 자세는 이동 명령인가, 다른 용도인가?	_____
2	이 앱이 특정 인물을 구분하는 방법은 무엇인가?	_____
3	Ackermann 로봇의 팔 동작 반응이 다른 레슨들과 다른 점은 무엇인가?	_____

심화 과제 — get_dif()의 판정 임계값(50)을 낮추거나 높여보고, 비슷한 색 옷을 입은 다른 사람과의 혼동이 얼마나 줄거나 느는지 실험해보세요.